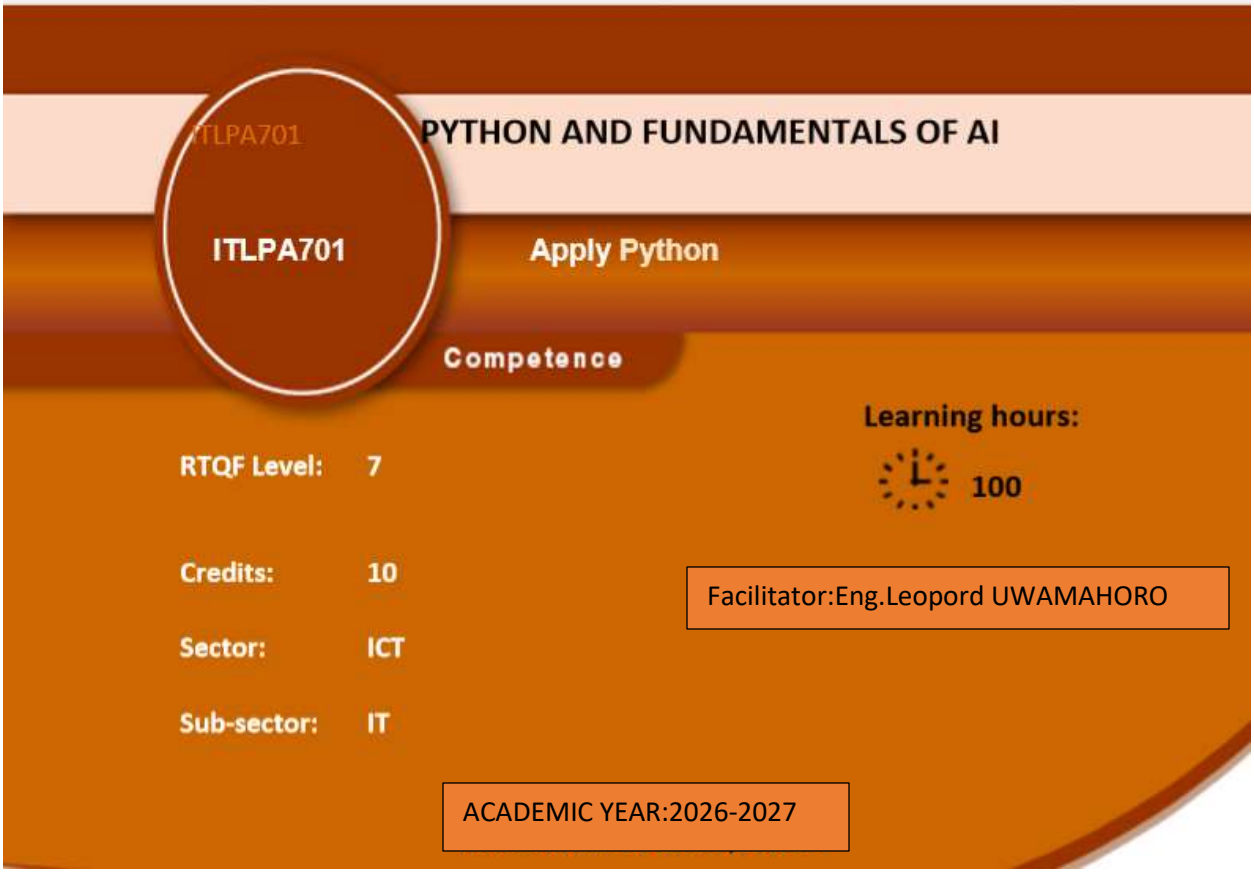
RWANDA POLYTECHNIC

NGOMA COLLEGE



Purpose statement

This module aims to introduce learners to the rudiments of applying Python programming language in various projects development, what makes it so massively popular, and its benefits and limitations. After completion of this module, learner will be able to apply python concept, developing AI based programs, web development (Server-side), software development, Data analysis as well as system scripting.

PAR1 :Python

LEARNING UNIT 1– PREPARE PYTHON ENVIRONMENT

The first step for every developer or data scientist who wants to work with Python is setting up an environment. Having the right tools, libraries, and dependencies to create and execute Python applications is ensured by a properly setup environment.

Learning Outcome 1.1: Identify Python Version

1.1.1 Definition of terms

High-level, adaptable, and well-liked, Python is a programming language that has many uses and is renowned for being easy to learn and understand. Since its initial release in 1991, Python—which was developed by Guido van Rossum—has grown to rank among the most popular programming languages worldwide.

Python is **an interpreted language**, which means that you can run code directly without the need for a separate compilation step. This makes it highly interactive and suitable for rapid development and prototyping.

Python comes with a **vast standard library** that provides ready-made modules and packages for a wide range of tasks, reducing the need to write code from scratch for common functionalities.

1.1.2 Use of python

Python is a versatile programming language used in a wide range of applications and industries. Here are some of the most common uses of Python .

- **Server-side**: Python is frequently used on the server-side to develop web applications, APIs, and backend services. It provides several frameworks and libraries that enable developers to create robust server-side applications.
- **Web Development**: Python web frameworks such as **Django, Flask, and Pyramid** are used.
- **API Development**: RESTful APIs can be created using frameworks like Flask or **Django REST framework,** allowing communication between various client applications and the server.
- **Machine Learning and AI APIs**:Server-side Python can provide APIs for machine learning models, enabling applications to make predictions and inferences using trained models.

- **Data Warehousing**: Python is used to manage and process data within data warehousing solutions, handling data extraction, transformation, and loading (ETL) processes.

- **Software development:**

Python is widely used in software development across various domains and applications. It's known for its readability, ease of use, and extensive library support, making it a versatile choice for software projects.

- **Desktop Applications**:Python can be used to create cross-platform desktop applications with libraries like PyQt, Tkinter, and Kivy.
- **Mobile App Development**: Python can be used for mobile app development, especially with the help of frameworks like Kivy and BeeWare.
- **Game Development:** Python can be used for developing 2D games and prototypes with libraries like Pygame.
- **Data Analysis and Visualization**:Python is widely used in data science and analytics. Libraries like NumPy, pandas, Matplotlib, and Seaborn are used for data manipulation and visualization, making it an excellent choice for software used in data-driven decision-making.
- **Machine Learning and Artificial Intelligence**:Python is a primary language for developing machine learning and AI applications. Frameworks like TensorFlow, Keras, and PyTorch are used to create and train machine learning models.

- **Mathematics**

Python is widely used in mathematics for a variety of purposes, including numerical computation, symbolic mathematics, data analysis, and visualization.

- **Numerical Computation**: Libraries like NumPy provide efficient numerical operations and support for multidimensional arrays, making it a valuable tool for mathematical. computations.
- **Plotting and Visualization**: Python has powerful libraries for data visualization, including Matplotlib, Seaborn, and Plotly.
- **Machine Learning in Mathematics**: Machine learning libraries such as scikit-learn can be used to apply mathematical techniques for pattern recognition, classification, and regression in various mathematical applications.

- **System scripting.**

Python is a popular choice for system scripting and automation tasks due to its versatility, ease of use, and cross-platform compatibility.

- File and Directory Operations: Python can be used to automate file and directory manipulation tasks, such as copying, moving, renaming, and deleting files. The os and shutil modules provide functions for these operations.

System Configuration: Python can be used to configure and manage system settings, including setting environment variables, managing configuration files, and automating system-wide settings changes.

- Backup and Restore: Python scripts can automate backup and restore processes for files, databases, and system configurations, helping ensure data integrity and availability.

1.1.3 Characteristic of python

The following characteristics collectively contribute to Python's popularity and effectiveness as a programming language.

- **Structure**

  Python is known for its clear and straightforward structure. It enforces indentation and relies on a consistent and readable syntax, making it easy to understand and maintain. Python uses **indentation (whitespace**) to define blocks of code, eliminating the need **for curly braces or other block delimiters** commonly found in other programming languages. This focus on readability and a well-defined structure makes Python an excellent choice for beginners and experienced programmers alike.

- **Application**

  Python is a general-purpose programming language, which means it can be used for a wide range of applications. It is not limited to a specific domain or purpose, making it versatile. Python is commonly used in web development, data science, artificial intelligence, scripting, automation, scientific computing, and more. Its diverse applications contribute to its popularity.

- **Checking**

  Python provides built-in features for error checking and debugging. It includes tools like exceptions, assertions, and debugging modules to help identify and handle errors in code. Python's error messages are often informative and guide developers in identifying the cause of issues in their code.

- **Integration**

  Python is known for its strong support for integration with other languages and systems. It has a well-defined C API, allowing it to be easily integrated with C and C++ code. Python can also interface with libraries and systems written in other languages, including Java, C#, and more. This integration capability is essential for using Python in a wide range of environments and with various external tools and resources.

1.1.3 Application of python

- **Flexibility:**Python is renowned for its flexibility, which allows it to be used in a wide range of applications.

- **Audience**:Python is suitable for a diverse audience, ranging from beginners to experienced developers, scientists, engineers, and more. Its readability and ease of use make it an excellent choice for those new to programming, while its powerful features and extensive libraries attract professionals in various domains.
- **Prerequisites**:
  Python's low entry barrier makes it accessible to a wide range of users. However, there are a few prerequisites to effectively work with Python:

  Basic Computer Skills: Familiarity with using computers and understanding file systems is useful.

  Installation: You need to install Python on your system. Most modern systems come with Python pre-installed, but you may need to install it if it's not available.

  Text Editor/IDE: Choose a text editor or integrated development environment (IDE) for coding in Python.

Learning Outcome 1.2: Add python directory

Preparing a Python environment is an essential step to set up a development or runtime environment for Python programming. A properly configured environment ensures that you have the necessary tools, libraries, and dependencies to write, test, and run Python code effectively.

1.2.1 **Local Environment set up**

he first step is to install Python itself. You can download the latest version of Python from the official website ,https://www.python.org/ or use a distribution like Anaconda, which includes many data science libraries.

Choose a Text Editor or Integrated Development Environment (IDE): You'll need a text editor or an integrated development environment to write and edit Python code. Some popular options include Visual Studio Code, PyCharm, Sublime Text, and Atom.

Setting Up the PATH Manually:

If you didn't check the "Add Python X.X to PATH" option during installation, you can add Python to the PATH manually:

- Open the Start menu and search for "Environment Variables."
- Click on "Edit the system environment variables."
- In the System Properties window, click the "Environment Variables" button.
- Under "System variables," scroll down to find "Path," and click "Edit."
- Click "New" and add the path to your Python installation directory (usually C:\Users\<your_username>\AppData\Local\Programs\Python\PythonXX).
- Click OK to save your changes.

After completing these steps, you should have Python set up on your Windows system, and you can run Python code either from the command line or from an integrated development environment like Visual Studio Code.

Learning Outcome 1.3: Identify Library

1.3.1 Here are definitions of key terms related to Python programming

Library : A library, in the context of programming, is a collection of **pre-written code and functions** that can be used by developers to perform common tasks. Libraries often provide a set of reusable functions, classes, and modules to simplify and accelerate software development. In Python, libraries are typically distributed as **packages**.

A package in Python is a way to organize and distribute collections of modules. Python's standard library includes many packages, and you can create your own packages to bundle related modules for easier code management and sharing.

In Python, the "**import**" statement is used to bring external modules, libraries, or packages into your Python code. Importing is essential for reusing code, and it enables you to extend Python's functionality by integrating external modules into your programs.

In the context of Python, **"install"** refers to the process of setting up a library, package, or module on your computer so that you can use it in your Python projects. This typically involves downloading the necessary code files, configuring dependencies, and making the package or library accessible for Python. Installation methods may vary, but they often include using package management tools like "pip" or manually installing from source code.

PIP stands for "**Pip Installs Packages**." It is a package manager for Python that simplifies the process of installing, upgrading, and managing external libraries and packages. You can use commands like "pip install package_name" to easily add new packages to your Python environment.

1.3.2 Getting library

To obtain a library using PIP (Python's package manager) and incorporate it into your Python project, you can follow these steps:

1. **Install PIP (if not already installed)**

PIP usually comes pre-installed with Python. To check if you have PIP installed, open a command prompt or terminal and run the following command:

```
pip --version
```

2. **Browsing the Python Package Index (PyPI)**

You can browse the Python Package Index (PyPI) website at https://pypi.org to search for libraries and packages. PyPI is a repository of Python software packages that you can easily install using PIP.

3. Download Library

You don't need to manually download libraries from PyPI when using PIP. PIP can automatically download and install packages from PyPI, which simplifies the process.

4. **Install Library**

To install a Python library using PIP, open a command prompt or terminal and use the following command

```
pip install library_name
```

Ex:
```
pip install requests
```

5. **Import Library**

Once the library is installed, you can import it into your Python code using the import statement.

```
import requests
```

6. **Use Library**

After importing the library, you can use its functions, classes, and modules in your Python code. The specific usage will depend on the library's documentation and the features it provides. For the "requests" library, which is commonly used for making HTTP requests, you might use it like this

```
import requests

response = requests.get("https://www.example.com")
print(response.text)
```

LEARNING UNIT 2–DEVELOP PYTHON CONCEPT

Learning Outcome 2.1: Writing python syntax

2.1.1 Execute syntax

Python syntax can be executed by writing directly in the Command Line

```
>>> print("Hello, World!")
Hello, World!
```

r by creating a python file on the server, **using the .py** file extension, and running it in the Command Line.

```
C:\Users\Your Name>python myfile.py
```

2.1.2 Python Indentation

Indentation refers to the spaces at the beginning of a code line.Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.
Python uses indentation to indicate a block of code.

Example

```
if 5 > 2:
  print("Five is greater than two!")
```

2.2.3 Use Command line

Command Line:

The command line, also known as a terminal or shell, is a text-based interface where you can interact with your computer by typing commands. In the context of Python, you can use the command line to run Python scripts, check Python version, install packages, and perform various tasks related to your Python environment.

Python Mode (Interactive Mode)

Python provides an interactive mode where you can enter Python code line by line and immediately see the results. You can access the Python interactive mode by opening your command prompt or terminal and typing python. This mode is useful for quick testing and experimentation.

Compile Program (No Compilation Required)

Python is an interpreted language, which means you don't need to compile your Python programs before running them. Instead, Python code is executed directly by the Python interpreter. This is different from languages like C++ or Java, which require compilation into machine code before execution.

Execute Program

To execute a Python program, you can save your code in a Python script (a text file with a .py extension) and run it using the Python interpreter. For example, if your program is in a file named my_program.py, you can execute it by running:

```
python my_program.py
```

Save Program

Saving a program in Python refers to saving your code in a text file with a .py extension. This allows you to reuse and execute your code later. You can save Python programs using a text editor or an integrated development environment (IDE). It's good practice to save your work regularly to avoid losing code.

2.2.4 Apply comments

Comments can be used to explain Python code.Comments can be used to make the code more readable.Comments can be used to prevent execution when testing code.

Creating a Comment

Comments starts with a #, and Python will ignore them.print("Hello, World!") #This is a comment

Multiline Comments

Python **does not** really have a syntax for multiline comments.To add a multiline comment you could insert a # for each line:

```
#This is a comment
#written in
#more than just one line
print("Hello, World!")
```

Learning Outcome 2.2: Perform declaration

Definition of Key terms

Python has no command for declaring a variable.A variable is created the **moment you first assign** a value to it.

```
x = 5
y = "John"
print(x)
print(y)
```

Variables do not need to be declared with any particular *type*, and can even change type after they have been set.

Example

```
x = 4       # x is of type int
x = "Sally" # x is now of type str
print(x)
```

Casting

If you want to specify the data type of a variable, this can be done with casting.

Example

```
x = str(3)    # x will be '3'
y = int(3)    # y will be 3
z = float(3)  # z will be 3.0
```

Get the Type

You can get the data type of a variable with the type() function.

Example

```
x = 5
y = "John"
print(type(x))
print(type(y))
```

Many Values to Multiple Variables

Python allows you to assign values to multiple variables in one line:

Example

```
x, y, z = "Orange", "Banana", "Cherry"
print(x)
print(y)
print(z)
```

Unpack a Collection

If you have a collection of values in a list, tuple etc. Python allows you to extract the values into variables. This is called *unpacking*.

Example

Unpack a list:

```
fruits = ["apple", "banana", "cherry"]
x, y, z = fruits
print(x)
print(y)
print(z)
```

Output Variables

The Python print() function is often used to output variables.In the print() function, you output multiple variables, separated by a comma:

Example

```
x = "Python"
y = "is"
z = "awesome"
print(x, y, z)
```

You can also use the + operator to output multiple variables:

Example

```
x = "Python "
y = "is "
z = "awesome"
print(x + y + z)
```

Notice In the print() function, when you try to combine a string and a number with the + operator, Python will give you an error.

Example

```
x = 5
y = "John"
print(x + y)
```

The best way to output multiple variables in the print() function is to separate them with commas, which even support different data types:

Example

```
x = 5
y = "John"
print(x, y)
```

Global Variables

Variables that are created **outside of a function** (as in all of the examples above) are known as global variables.

Global variables can be used by everyone, both inside of **functions and outside**.

Example

Create a variable outside of a function, and use it inside the function

```
x = "awesome"

def myfunc():
  print("Python is " + x)

myfunc()
```

If you create a variable with the same name inside a function, this variable will be local, and can only be **used inside the function**. The global variable with thesame name will remain as it was, global and with the original value.

Example

Create a variable inside a function, with the same name as the global variable

```
x = "awesome"

def myfunc():
  x = "fantastic"
  print("Python is " + x)

myfunc()

print("Python is " + x)
```

The global Keyword

Normally, when you create a variable inside a function, that variable is local, and can only be used inside that function.

To create a global variable inside a function, you can use the global keyword.
Example

If you use the global keyword, the variable belongs to the global scope:

```
def myfunc():
  global x
  x = "fantastic"

myfunc()

print("Python is " + x)
```

Also, use the global keyword if you want to change a global variable inside a function.
Example

To change the value of a global variable inside a function, refer to the variable by using the global keyword:

```
x = "awesome"

def myfunc():
  global x
  x = "fantastic"

myfunc()

print("Python is " + x)
```

Learning Outcome 2.3: Differentiate data type

In programming, data type is an important concept.Variables can store data of different types, and different types can do different things.

2.3.1 Define build-in data type

Python has the following data types built-in by default, in these categories:

Text Type: str

Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	dict
Set Types:	set, frozenset
Boolean Type:	bool
Binary Types:	bytes, bytearray, memoryview
None Type:	NoneType

2.3.2 Numbers

Python Numbers

There are three numeric types in Python:

- int
- float
- complex

Variables of numeric types are created when you assign a value to them:

Example

```
x = 1    # int
y = 2.8  # float
z = 1j   # complex
```

To verify the type of any object in Python, use the type() function:

Example

```
print(type(x))
print(type(y))
print(type(z))
```

Int

Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

Example

Integers:

```
x = 1
y = 35656222554887711
z = -3255522
```

Float

Float, or "floating point number" is a number, positive or negative, containing one or more decimals.

Example

Floats:

```
x = 1.10
y = 1.0
z = -35.59
```

Complex

Complex numbers are written with a "j" as the imaginary part:

Example

Complex:

```
x = 3+5j y = 5j
z = -5j
```

Type Conversion

You can convert from one type to another with the int(), float(), and complex() methods:

Example:Convert from one type to another:

```
x = 1    # int
y = 2.8  # float
z = 1j   # complex

#convert from int to float:
a = float(x)

#convert from float to int:
b = int(y)

#convert from int to complex:
c = complex(x)
```

Note: You cannot convert complex numbers into another number type.

Random Number

Python does not have a random() function to make a random number, but Python has a built-in module called random that can be used to make random numbers:

Example:Import the random module, and display a random number between 1 and 9:

```
import random
print(random.randrange(1, 10))
```

Strings

Strings in python are surrounded by either single quotation marks, or double quotation marks.

'hello' is the same as "hello".You can display a string literal with the print() function:

Example

```
print("Hello")
print('Hello')
```

Assign String to a Variable:Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

Example

```
a = "Hello"
```

Multiline Strings:You can assign a multiline string to a variable by using three quotes:

Strings are Arrays

Like many other popular programming languages, strings in Python are arrays of bytes representing unicode characters.

However, Python does not have a character data type, a single character is simply a string with a length of 1.

Square brackets can be used to access elements of the string.

Example

Get the character at position 1 (remember that the first character has the position 0):

```
>>> a="Hello"
>>> print(a[2])
l
>>> print(a[4])
o
>>>
```

Looping Through a String

Since strings are arrays, we can loop through the characters in a string, with a for loop.

Example

Loop through the letters in the word "banana":

```
for x in "hello":
  print(x)
```

String Length

To get the length of a string, use the len() function.

Example
The len() function returns the length of a string:

```
a = "Hello"
print(len(a))
```

```
>>> print(len(a))
5
```

Use it in an if statement:

Example

Print only if "free" is present:

```
txt = "The best things in life are free!"
if "free" in txt:
  print("Yes, 'free' is present.")
```

Check if NOT

To check if a certain phrase or character is NOT present in a string, we can use the keyword not in.

Example

Check if "expensive" is NOT present in the following text:

```
txt = "The best things in life are free!"
print("expensive" not in txt)
```

Example

print only if "expensive" is NOT present:

```
txt = "The best things in life are free!"
if "expensive" not in txt:
  print("No, 'expensive' is NOT present.")
```

Slicing

You can return a range of characters by using the slice syntax.Specify the start index and the end index, separated by a colon, to return a part of the string.

Example

Get the characters from position 2 to position 5 (not included):

```
b = "Hello, World!"
print(b[3:7])
```

Note: The first character has index 0.

Slice From the Start

By leaving out the start index, the range will start at the first character:

Example

Get the characters from the start to position 5 (not included):

```
b = "Hello, World!"
print(b[:5])
```

Slice To the End

By leaving out the end index, the range will go to the end:

Example

Get the characters from position 2, and all the way to the end:

```
b = "Hello, World!"
print(b[2:])
```

Negative Indexing

Use negative indexes to start the slice from the end of the string:

Example

Get the characters:

From: "o" in "World!" (position -5)

To, but not included: "d" in "World!" (position -2):

```
b = "Hello, World!"
print(b[-5:-2])
```

Python - Modify Strings

Example

The upper() method returns the string in upper case:

```
a = "Hello, World!"
print(a.upper())
```

Lower Case

Example

The lower() method returns the string in lower case:

```
a = "Hello, World!"
print(a.lower())
```

Remove Whitespace

Whitespace is the space before and/or after the actual text, and very often you want to remove this space.

Example

The strip() method removes any whitespace from the beginning or the end:

```
a = " Hello, World! "
print(a.strip()) # returns "Hello, World!"
```

Python - String Concatenation

To concatenate, or combine, two strings you can use the + operator.

Example

Merge variable a with variable b into variable c:

```
a = "Hello"
b = "World"
c = a + b
print(c)
```

Python - Format – Strings

We cannot combine strings and numbers like this:
Example
```
age = 36
txt = "My name is John, I am " + age
print(txt)
```

But we can combine strings and numbers by using the format() method!

The format() method takes the passed arguments, formats them, and places them in the string where the placeholders {} are:
Example

Use the format() method to insert numbers into strings:
```
age = 36
txt = "My name is John, and I am {}"
print(txt.format(age))
```

The format() method takes unlimited number of arguments, and are placed into the respective placeholders:
Example
```
quantity = 3
itemno = 567
price = 49.95
myorder = "I want {} pieces of item {} for {} dollars."
print(myorder.format(quantity, itemno, price))
```

You can use index numbers {0} to be sure the arguments are placed in the correct placeholders:
Example
```
quantity = 3
itemno = 567
price = 49.95
myorder = "I want to pay {2} dollars for {0} pieces of item {1}."
print(myorder.format(quantity, itemno, price))
```
Python - Escape Characters
To insert characters that are illegal in a string, use an escape character.An escape character is a backslash \ followed by the character you want to insert.An example of an illegal character is a double quote inside a string that is surrounded by double quotes:
Example
You will get an error if you use double quotes inside a string that is surrounded by double quotes:

txt = "We are the so-called "Vikings" from the north."

Example

The escape character allows you to use double quotes when you normally would not be allowed:

txt = "We are the so-called \"Vikings\" from the north."

Other escape characters used in Python:

Code	Result
\'	Single Quote
\\	Backslash
\n	New Line
\r	Carriage Return
\t	Tab
\b	Backspace
\ooo	Octal value
\xhh	Hex value

Syntax

```
txt = 'It\'s alright.'
print(txt)
```

```
txt = "This will insert one \\ (backslash)."
print(txt)
```

```
txt = "Hello\nWorld!"
print(txt)
```

```
txt = "Hello\rWorld!"
print(txt)
```

```
txt = "Hello\tWorld!"
print(txt)
```

```
#This example erases one character (backspace):
txt = "Hello \bWorld!"
print(txt)
```

```
#A backslash followed by three integers will result in a octal value
txt = "\110\145\154\154\157"
print(txt)
#A backslash followed by an 'x' and a hex number represents a hex value
txt = "\x48\x65\x6c\x6c\x6f"
print(txt)
```

String Methods

Python has a set of built-in methods that you can use on strings.

Method	Description
capitalize()	Converts the first character to upper case
casefold()	Converts string into lower case
center()	Returns a centered string
count()	Returns the number of times a specified value occurs in a string
encode()	Returns an encoded version of the string
endswith()	Returns true if the string ends with the specified value
expandtabs()	Sets the tab size of the string
find()	Searches the string for a specified value and returns the position of where it was found
format()	Formats specified values in a string
format_map()	Formats specified values in a string
index()	Searches the string for a specified value and returns the position of where it was found
isalnum()	Returns True if all characters in the string are alphanumeric

isalpha()	Returns True if all characters in the string are in the alphabet
isascii()	Returns True if all characters in the string are ascii characters
isdecimal()	Returns True if all characters in the string are decimals
isdigit()	Returns True if all characters in the string are digits
isidentifier()	Returns True if the string is an identifier
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric
isprintable()	Returns True if all characters in the string are printable
isspace()	Returns True if all characters in the string are whitespaces
istitle()	Returns True if the string follows the rules of a title
isupper()	Returns True if all characters in the string are upper case
join()	Joins the elements of an iterable to the end of the string
ljust()	Returns a left justified version of the string
lower()	Converts a string into lower case
lstrip()	Returns a left trim version of the string
maketrans()	Returns a translation table to be used in translations
partition()	Returns a tuple where the string is parted into three parts
replace()	Returns a string where a specified value is replaced with a specified value
rfind()	Searches the string for a specified value and returns the last position of where it was found
rindex()	Searches the string for a specified value and returns the last position of where it was found
rjust()	Returns a right justified version of the string
rpartition()	Returns a tuple where the string is parted into three parts
rsplit()	Splits the string at the specified separator, and returns a list
rstrip()	Returns a right trim version of the string
split()	Splits the string at the specified separator, and returns a list
splitlines()	Splits the string at line breaks and returns a list

startswith()	Returns true if the string starts with the specified value
strip()	Returns a trimmed version of the string
swapcase()	Swaps cases, lower case becomes upper case and vice versa
title()	Converts the first character of each word to upper case
translate()	Returns a translated string
upper()	Converts a string into upper case
zfill()	Fills the string with a specified number of 0 values at the beginning

LEARNING UNIT 3 – DEVELOP PYTHON SCRIPT

A Python script is a sequence of Python programming instructions contained within a text file. These instructions, written in the Python programming language, are typically used to automate tasks, perform specific operations, or execute a series of actions. Python scripts can be simple, consisting of just a few lines of code, or they can be complex applications with thousands of lines of code.

Python divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators

- Bitwise operators

Python Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

Python Assignment Operators

Assignment operators are used to assign values to variables:

1. **Assignment Operator (=)**: The equal sign (`=`) is the basic assignment operator, which assigns the value on the right to the variable on the left.

```python
x = 10  # Assigns the value 10 to the variable x
```

2. **Addition Assignment Operator (+=)**: This operator adds the right operand to the left operand and assigns the result to the left operand.

```python
x = 5
x += 3  # Equivalent to x = x + 3, assigns 8 to x
```

3. **Subtraction Assignment Operator (-=)**: This operator subtracts the right operand from the left operand and assigns the result to the left operand.

```python
x = 10
x -= 4  # Equivalent to x = x - 4, assigns 6 to x
```

4. **Multiplication Assignment Operator (*=)**: This operator multiplies the left operand by the right operand and assigns the result to the left operand.

```python
x = 7
x *= 2  # Equivalent to x = x * 2, assigns 14 to x
```

5. **Division Assignment Operator (/=)**: This operator divides the left operand by the right operand and assigns the result to the left operand.

```python
x = 12
x /= 3  # Equivalent to x = x / 3, assigns 4.0 to x
```

6. **Modulus Assignment Operator (%=)**: This operator calculates the remainder when the left operand is divided by the right operand and assigns the result to the left operand.

```python
x = 20
x %= 7  # Equivalent to x = x % 7, assigns 6 to x
```

7. **Exponentiation Assignment Operator (=)**: This operator raises the left operand to the power of the right operand and assigns the result to the left operand.

```python
x = 2
x **= 3  # Equivalent to x = x ** 3, assigns 8 to x
```

8. **Floor Division Assignment Operator (//=)**: This operator performs floor division (integer division) between the left operand and the right operand and assigns the result to the left operand.

```python
x = 15
x //= 4  # Equivalent to x = x // 4, assigns 3 to x
```

These assignment operators are useful for simplifying and making code more concise by combining operations and assignments into a single step.

Python Comparison Operators

Comparison operators are used to compare two values:

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Python Logical Operators

Logical operators are used to combine conditional statements

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

```
x = 5
print(x > 3 and x < 10)
```

```
x = 5
print(x > 3 or x < 4)
```

```
x = 5
print(not(x > 3 and x < 10))
```

Operator	Name	Description	Example
&	AND	Sets each bit to 1 if both bits are 1	x & y
|	OR	Sets each bit to 1 if one of two bits is 1	x | y
^	XOR	Sets each bit to 1 if only one of two bits is 1	x ^ y
~	NOT	Inverts all the bits	~x
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off	x << 2
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	x >> 2

(The & operator compares each bit and set it to 1 if both are 1, otherwise it is set to 0:)

```
print(6 & 3)
returns 2
```

```
6 = 0000000000000110
3 = 0000000000000011
--------------------
2 = 0000000000000010
```

(The | operator compares each bit and set it to 1 if one of two is 1, otherwise it is set to 0:)

```
print(6 | 3)
return 7
```

```
6 = 0000000000000110
3 = 0000000000000011
--------------------
7 = 0000000000000111
```

(The ^ operator compares each bit and set it to 1 if only one of two is 1, otherwise it is set to 0:)

```
return 7
```

(The ~ operator inverts all bits. Example: 0 becomes 1, and 1 becomes 0.)

(The << operator inserts the specified number of 0's (in this case 2) from the right and let the same amount of leftmost bits fall off:)

```
return 12
print(3 << 2)
```

```
If you push 00 in from the left:
 3 = 0000000000000011
becomes
12 = 0000000000001100
```

(The >> operator moves each bit the specified number of times to the right. Empty holes at the left are filled with 0's.)

```
print(8 >> 2)
return 2
```

```
If you move each bit 2 times to the right, 8 becomes
 8 = 0000000000001000
becomes
```

Python Membership Operators

Membership operators are used to test if a sequence is presented in an object

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

Python Bitwise Operators

Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description	Example
&	AND	Sets each bit to 1 if both bits are 1	x & y
|	OR	Sets each bit to 1 if one of two bits is 1	x | y
^	XOR	Sets each bit to 1 if only one of two bits is 1	x ^ y
~	NOT	Inverts all the bits	~x
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off	x << 2
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	x >> 2

(The & operator compares each bit and set it to 1 if both are 1, otherwise it is set to 0:)

```
print(6 & 3)
```
retuns 2

```
6 = 0000000000000110
3 = 0000000000000011
--------------------
2 = 0000000000000010
```

```
print(6 | 3)
```
return 7

```
6 = 0000000000000110
3 = 0000000000000011
--------------------
7 = 0000000000000111
```
return 7

(The << operator inserts the specified number of 0's (in this case 2) from the right and let the same amount of leftmost bits fall off:)

```
print(3 << 2)
```
return 12

```
If you push 00 in from the left:
 3 = 0000000000000011
becomes
12 = 0000000000001100
```

The >> operator moves each bit the specified number of times to the right. Empty holes at the left are filled with 0's.

```
print(8 >> 2)
```
return 2

```
If you move each bit 2 times to the right, 8 becomes
 8 = 0000000000001000
becomes
```

Python Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

Learning Outcome 3.2: Determine collection of data

3.2.1 Definition of list
Python Lists

```
mylist = ["apple", "banana", "cherry"]
```

List
Lists are used to store multiple items in a single variable.
Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage.
Lists are created using square brackets:
Example
Create a List:

```
thislist = ["apple", "banana", "cherry"]
print(thislist)
```

List Items

List items are **ordered, changeable, and allow** duplicate values.List items are indexed, the first item has index [0], the second item has index [1] etc.
Ordered

When we say that lists are ordered, it means that the items have a defined order, and that order will not change.If you add new items to a list, the new items will be placed at the end of the list.
Changeable

The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.
Allow Duplicates
Since lists are indexed, lists can have items with the same value:
Example
Lists allow duplicate values:

```
thislist = ["apple", "banana", "cherry", "apple", "cherry"]
print(thislist)
```

List Length

To determine how many items a list has, use the len() function:

Example: Print the number of items in the list:

```
thislist = ["apple", "banana", "cherry"]
print(len(thislist))
```

List Items - Data Types

List items can be of any data type:

Example

String, int and boolean data types:

```
list1 = ["apple", "banana", "cherry"]
list2 = [1, 5, 7, 9, 3]
list3 = [True, False, False]
```

A list can contain different data types:

Example

A list with strings, integers and boolean values:

```
list1 = ["abc", 34, True, 40, "male"]
```

From Python's perspective, lists are defined as objects with the data type 'list':

<class 'list'>

Example

What is the data type of a list?

```
mylist = ["apple", "banana", "cherry"]
print(type(mylist))
```

The list() Constructor

It is also possible to use the list() constructor when creating a new list.

Example

Using the list() constructor to make a List:

```
thislist = list(("apple", "banana", "cherry")) # note the double round-brackets
print(thislist)
```

Access item

List items are indexed and you can access them by referring to the index number:

Example

Print the second item of the list:

```
thislist = ["apple", "banana", "cherry"]
print(thislist[1])
```

Note: The first item has index 0.

Range of Indexes

You can specify a range of indexes by specifying where to start and where to end the range.

When specifying a range, the return value will be a new list with the specified items.

Example

Return the third, fourth, and fifth item:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:5])
```

By leaving out the start value, the range will start at the first item:

Example

This example returns the items from the beginning to, but NOT including, "kiwi":

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[:4])
```

By leaving out the end value, the range will go on to the end of the list:

Example

This example returns the items from "cherry" to the end:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:])
```

Range of Negative Indexes

Specify negative indexes if you want to start the search from the end of the list:

Example

This example returns the items from "orange" (-4) to, but NOT including "mango" (-1):

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[-4:-1])
```

Check if Item Exists

To determine if a specified item is present in a list use the in keyword:

Example

Check if "apple" is present in the list:

```
thislist = ["apple", "banana", "cherry"]
if "apple" in thislist:
  print("Yes, 'apple' is in the fruits list")
```

Change Item Value:

To change the value of a specific item, refer to the index number:

Example

Change the second item:

```
thislist = ["apple", "banana", "cherry"]
thislist[1] = "blackcurrant"
print(thislist)
```

Change a Range of Item Values

To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to insert the new values:

Example

Change the values "banana" and "cherry" with the values "blackcurrant" and "watermelon":

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
thislist[1:3] = ["blackcurrant", "watermelon"]
print(thislist)
```

If you insert *more* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

Example

Change the second value by replacing it with *two* new values:

```
thislist = ["apple", "banana", "cherry"]
thislist[1:2] = ["blackcurrant", "watermelon"]
print(thislist).
```

Note: The length of the list will change when the number of items inserted does not match the number of items replaced.

If you insert *less* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

Example

Change the second and third value by replacing it with *one* value:

```
thislist = ["apple", "banana", "cherry"]
thislist[1:3] = ["watermelon"]
print(thislist)
```

Insert Items

To insert a new list item, without replacing any of the existing values, we can use the insert() method.

The insert() method inserts an item at the specified index:

Example

Insert "watermelon" as the third item:

```
thislist = ["apple", "banana", "cherry"]
thislist.insert(2, "watermelon")
print(thislist)
```

Add List Items

Append Items

To add an item to the end of the list, use the append() method:

Example

Using the append() method to append an item:

```
thislist = ["apple", "banana", "cherry"]
thislist.append("orange")
print(thislist)
```

Insert Items

To insert a list item at a specified index, use the insert() method.

The insert() method inserts an item at the specified index:

Example

Insert an item as the second position:

```
thislist = ["apple", "banana", "cherry"]
thislist.insert(1, "orange")
print(thislist)
```

Insert Items

To insert a list item at a specified index, use the insert() method.

The insert() method inserts an item at the specified index:

Example

Insert an item as the second position:

```
thislist = ["apple", "banana", "cherry"]
thislist.insert(1, "orange")
print(thislist)
```

Add items

Remove Specified Item

The remove() method removes the specified item.

Example

Remove "banana":

```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
print(thislist
```

If there are more than one item with the specified value, the remove() method removes the first occurance:

Example

Remove the first occurance of "banana":

```
thislist = ["apple", "banana", "cherry", "banana", "kiwi"]
thislist.remove("banana")
print(thislist)
```

Remove Specified Index

The pop() method removes the specified index.

Example
Remove the second item:

```
thislist = ["apple", "banana", "cherry"]
thislist.pop(1)
print(thislist)
```

If you do not specify the index, the pop() method removes the last item.

Example

Remove the last item:

```
thislist = ["apple", "banana", "cherry"]
thislist.pop()
print(thislist)
```

The del keyword also removes the specified index:

Example

Remove the first item:

```
thislist = ["apple", "banana", "cherry"]
del thislist[0]
print(thislist)
```

The del keyword can also delete the list completely.

Example

Delete the entire list:

```
thislist = ["apple", "banana", "cherry"]
del thislist
```

Clear the List

The clear() method empties the list.

The list still remains, but it has no content.

Example

Clear the list content:

```
thislist = ["apple", "banana", "cherry"]
thislist.clear()
print(thislist)
```

Copy a List

You cannot copy a list simply by typing list2 = list1, because: list2 will only be a *reference* to list1, and changes made in list1 will automatically also be made in list2.

There are ways to make a copy, one way is to use the built-in List method copy().

Example

Make a copy of a list with the copy() method:

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist.copy()
print(mylist)
```

Another way to make a copy is to use the built-in method list().

Example

Make a copy of a list with the list() method:

```
thislist = ["apple", "banana", "cherry"]
mylist = list(thislist)
print(mylist)
```

Sort List Alphanumerically

List objects have a sort() method that will sort the list alphanumerically, ascending, by default:

Example

Sort the list alphabetically:

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort(
```

Sort Descending

To sort descending, use the keyword argument reverse = True:

Example

Sort the list descending:

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort(reverse = True)
print(thislist)
```

Sort the list descending:

```
thislist = [100, 50, 65, 82, 23]
thislist.sort(reverse = True)
print(thislist)
```

Join Two Lists

There are several ways to join, or concatenate, two or more lists in Python.

One of the easiest ways are by using the + operator.

Example

Join two list:

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]

list3 = list1 + list2
print(list3)
```

Another way to join two lists is by appending all the items from list2 into list1, one by one:

Example

Append list2 into list1:

```
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]

for x in list2:
  list1.append(x)

print(list1)
```

Or you can use the extend() method, where the purpose is to add elements from one list to another list:

Example

Use the extend() method to add list2 at the end of list1:

```
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]

list1.extend(list2)
print(list1)
```

List Methods

Python has a set of built-in methods that you can use on lists.

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

3.2.2 Definition of Tuple

A tuple is a collection which is ordered and **unchangeable**.Tuples are written with round brackets.
Example
Create a Tuple:

```
thistuple = ("apple", "banana", "cherry")
print(thistuple)
```

Tuple Items
Tuple items are ordered, unchangeable, and allow duplicate values.Tuple items are indexed, the first item has index [0], the second item has index [1] etc.
Ordered
When we say that tuples are ordered, it means that the items have a defined order, and that order will not cha
Unchangeable
Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

Allow Duplicates

Since tuples are indexed, they can have items with the same value:

Example

Tuples allow duplicate values:

```
thistuple = ("apple", "banana", "cherry", "apple", "cherry")
print(thistuple)
```

Tuple Length

To determine how many items a tuple has, use the len() function:

Example

Print the number of items in the tuple:

```
thistuple = ("apple", "banana", "cherry")
print(len(thistuple))
```

Create Tuple With One Item

To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

Example

One item tuple, remember the comma:

```
thistuple = ("apple",)
print(type(thistuple))

#NOT a tuple
thistuple = ("apple")
print(type(thistuple))
```

Tuple Items - Data Types

Tuple items can be of any data type:

Example

String, int and boolean data types:

```
tuple1 = ("apple", "banana", "cherry")
tuple2 = (1, 5, 7, 9, 3)
tuple3 = (True, False, False)
```

A tuple can contain different data types:

Example

A tuple with strings, integers and boolean values:

```
tuple1 = ("abc", 34, True, 40, "male")
```

type()

From Python's perspective, tuples are defined as objects with the data type 'tuple':

<class 'tuple'>

Example

What is the data type of a tuple?

```
mytuple = ("apple", "banana", "cherry")
print(type(mytuple))
```

The tuple() Constructor

It is also possible to use the tuple() constructor to make a tuple.

Example

Using the tuple() method to make a tuple:

```
thistuple = tuple(("apple", "banana", "cherry")) # note the double round-brackets
print(thistuple)
```

Access Tuple Items

You can access tuple items by referring to the index number, inside square brackets:

Example

Print the second item in the tuple:

```
thistuple = ("apple", "banana", "cherry")
print(thistuple[1])
```

Negative Indexing

Negative indexing means start from the end.

-1 refers to the last item, -2 refers to the second last item etc.

Example

Print the last item of the tuple:

```
thistuple = ("apple", "banana", "cherry")
print(thistuple[-1])
```

Range of Indexes

You can specify a range of indexes by specifying where to start and where to end the range.

When specifying a range, the return value will be a new tuple with the specified items.

Example

Return the third, fourth, and fifth item:

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[2:5])
```

Example

This example returns the items from the beginning to, but NOT included, "kiwi":

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[:4])
```

By leaving out the end value, the range will go on to the end of the list:

Example

This example returns the items from "cherry" and to the end:

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[2:])
```

Range of Negative Indexes

Specify negative indexes if you want to start the search from the end of the tuple:

Example

This example returns the items from index -4 (included) to index -1 (excluded)

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[-4:-1])
```

Check if Item Exists

To determine if a specified item is present in a tuple use the in keyword:

Example

Check if "apple" is present in the tuple:

```
thistuple = ("apple", "banana", "cherry")
if "apple" in thistuple:
  print("Yes, 'apple' is in the fruits tuple")
```

Change Tuple Values

Once a tuple is created, you cannot change its values. Tuples are unchangeable, or immutable as it also is called.

But there is a workaround. You can convert the tuple into a list, change the list, and convert the list back into a tuple.

Example

Convert the tuple into a list to be able to change it:

```
x = ("apple", "banana", "cherry")
y = list(x)
y[1] = "kiwi"
x = tuple(y)

print(x)
```

Add Items

Since tuples are immutable, they do not have a built-in append() method, but there are other ways to add items to a tuple.

1. **Convert into a list**: Just like the workaround for *changing* a tuple, you can convert it into a list, add your item(s), and convert it back into a tuple.

Example

Convert the tuple into a list, add "orange", and convert it back into a tuple:

```
thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.append("orange")
thistuple = tuple(y)
```

2. **Add tuple to a tuple**. You are allowed to add tuples to tuples, so if you want to add one item, (or many), create a new tuple with the item(s), and add it to the existing tuple:

Example

Create a new tuple with the value "orange", and add that tuple:

```
thistuple = ("apple", "banana", "cherry")
y = ("orange",)
thistuple += y

print(thistuple)
```

Note: You cannot remove items in a tuple.

Tuples are **unchangeable**, so you cannot remove items from it, but you can use the same workaround as we used for changing and adding tuple items:

Example

Convert the tuple into a list, remove "apple", and convert it back into a tuple:

```
thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.remove("apple")
thistuple = tuple(y)
```

Or you can delete the tuple completely:

Example

The del keyword can delete the tuple completely:

```
thistuple = ("apple", "banana", "cherry")
del thistuple
print(thistuple) #this will raise an error because the tuple no longer exists
```

Join Two Tuples

To join two or more tuples you can use the + operator:

Example

Join two tuples:

```
tuple1 = ("a", "b" , "c")
tuple2 = (1, 2, 3)

tuple3 = tuple1 + tuple2
print(tuple3)
```

Example

Multiply the fruits tuple by 2:

```
fruits = ("apple", "banana", "cherry")
mytuple = fruits * 2
```

```
print(mytuple)
```

Loop Through the Index Numbers

You can also loop through the tuple items by referring to their index number.

Use the range() and len() functions to create a suitable iterable.

Example

Print all items by referring to their index number:

```
thistuple = ("apple", "banana", "cherry")
for i in range(len(thistuple)):
  print(thistuple[i])
```

Using a While Loop

You can loop through the tuple items by using a while loop.

Use the len() function to determine the length of the tuple, then start at 0 and loop your way through the tuple items by referring to their indexes.

Remember to increase the index by 1 after each iteration.

Example

Print all items, using a while loop to go through all the index numbers:

```
thistuple = ("apple", "banana", "cherry")
i = 0
while i < len(thistuple):
  print(thistuple[i])
  i = i + 1
```

Tuple Methods

Python has two built-in methods that you can use on tuples.

Method	Description
count()	Returns the number of times a specified value occurs in a tuple
index()	Searches the tuple for a specified value and returns the position of where it was found

3.2.3 Definition of Set

Sets are used to store multiple items in a single variable. A set is a collection which is ***unordered, unchangeable*,*** **and** ***unindexed***.

*** Note:** Set *items* are unchangeable, but you can remove items and add new items.

Sets are written with curly brackets.

Example

Create a Set:

```
thisset = {"apple", "banana", "cherry"}
print(thisset)
```

Duplicates Not Allowed

Sets cannot have two items with the same value.

Example

Duplicate values will be ignored:

```
thisset = {"apple", "banana", "cherry", "apple"}

print(thisset)
```

Note: The values True and 1 are considered the same value in sets, and are treated as duplicates:

Example

True and 1 is considered the same value:

```
thisset = {"apple", "banana", "cherry", True, 1, 2}

print(thisset)
```

Note: The values False and 0 are considered the same value in sets, and are treated as duplicates:

Example

False and 0 is considered the same value:

```
thisset = {"apple", "banana", "cherry", False, True, 0}

print(thisset)
```

Set Items - Data Types

Set items can be of any data type:

Example

String, int and boolean data types:

```
set1 = {"apple", "banana", "cherry"}
set2 = {1, 5, 7, 9, 3}
set3 = {True, False, False}
```

The set() Constructor

It is also possible to use the set() constructor to make a set.

Example

Using the set() constructor to make a set:

```
thisset = set(("apple", "banana", "cherry")) # note the double round-brackets
print(thisset)
```

Access Items

You cannot access items in a set by referring to an index or a key.But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the in keyword.

Example

Loop through the set, and print the values:

```
thisset = {"apple", "banana", "cherry"}

for x in thisset:
  print(x)
```

Example

Check if "banana" is present in the set:

```
thisset = {"apple", "banana", "cherry"}

print ("banana" in thisset)
```

Change Items

Once a set is created, you cannot change its items, but you can add new items.

Add Items

Once a set is created, you cannot change its items, but you can add new items.

To add one item to a set use the add() method.

Example

Add an item to a set, using the add() method:

```
thisset = {"apple", "banana", "cherry"}

thisset.add("orange")

print(thisset)
```

Add Sets

To add items from another set into the current set, use the update() method.

Example

Add elements from tropical into thisset:

```
thisset = {"apple", "banana", "cherry"}
tropical = {"pineapple", "mango", "papaya"}

thisset.update(tropical)

print(thisset)
```

Add Any Iterable

The object in the update() method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).

Example

Add elements of a list to at set:

```
thisset = {"apple", "banana", "cherry"}
mylist = ["kiwi", "orange"]

thisset.update(mylist)

print(thisset)
```

Add Any Iterable

The object in the update() method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).

Remove Item

To remove an item in a set, use the remove(), or the discard() method.

Example

Remove "banana" by using the remove() method:

```
thisset = {"apple", "banana", "cherry"}

thisset.remove("banana")

print(thisset)
```

Example

Remove "banana" by using the discard() method:

```
thisset = {"apple", "banana", "cherry"}

thisset.discard("banana")

print(thisset)
```

Note: If the item to remove does not exist, discard() will **NOT** raise an error.

You can also use the pop() method to remove an item, but this method will remove a random item, so you cannot be sure what item that gets removed.

The return value of the pop() method is the removed item.

Example

Remove a random item by using the pop() method:

```
thisset = {"apple", "banana", "cherry"}

x = thisset.pop()

print(x)

print(thisset)
```

Note: Sets are *unordered*, so when using the pop() method, you do not know which item that gets removed.

Example

The **clear()** method empties the set:

```
thisset = {"apple", "banana", "cherry"}

thisset.clear()

print(thisset)
```

Example

The del keyword will delete the set completely:

```
thisset = {"apple", "banana", "cherry"}

del thisset

print(thisset)
```

Join Two Sets

There are several ways to join two or more sets in Python.You can use the union() method that returns a new set containing all items from both sets, or the update() method that inserts all the items from one set into another:

Example

The **union()** method returns a new set with all items from both sets:

```
set1 = {"a", "b" , "c"}
set2 = {1, 2, 3}

set3 = set1.union(set2)
print(set3)
```

Example

The update() method inserts the items in set2 into set1:

```
set1 = {"a", "b" , "c"}
set2 = {1, 2, 3}
```

```
set1.update(set2)
print(set1)
```

Note: Both union() and update() will exclude any duplicate items.

Keep ONLY the Duplicates

The intersection_update() method will keep only the items that are present in both sets.

Example

Keep the items that exist in both set x, and set y:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}

x.intersection_update(y)

print(x
```

The intersection() method will return a *new* set, that only contains the items that are present in both sets.

Example

Return a set that contains the items that exist in both set x, and set y:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}

z = x.intersection(y)
```

Keep All, But NOT the Duplicates

The symmetric_difference_update() method will keep only the elements that are NOT present in both sets.

Example

Keep the items that are not present in both sets:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}

x.symmetric_difference_update(y)

print(x)
```

The symmetric_difference() method will return a new set, that contains only the elements that are NOT present in both sets.

Example

Return a set that contains all items from both sets, except items that are present in both:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}

z = x.symmetric_difference(y)

print(z)
```

Note: The values True and 1 are considered the same value in sets, and are treated as duplicates:

Example

True and 1 is considered the same value:

```
x = {"apple", "banana", "cherry", True}
y = {"google", 1, "apple", 2}

z = x.symmetric_difference(y)

print(z)
```

Set Methods

Python has a set of built-in methods that you can use on sets.

Method	Description
add()	Adds an element to the set
clear()	Removes all the elements from the set
copy()	Returns a copy of the set
difference()	Returns a set containing the difference between two or more sets
difference_update()	Removes the items in this set that are also included in another, specified set
discard()	Remove the specified item
intersection()	Returns a set, that is the intersection of two other sets
intersection_update()	Removes the items in this set that are not present in other, specified set(s)
issubset()	Returns whether another set contains this set or not
issuperset()	Returns whether this set contains another set or not
pop()	Removes an element from the set
remove()	Removes the specified element
symmetric_difference()	Returns a set with the symmetric differences of two sets
symmetric_difference_update()	inserts the symmetric differences from this set and another

union()	Return a set containing the union of sets
update()	Update the set with the union of this set and others

Dictionary

Dictionaries are used to store data values in key:value pairs.A dictionary is a collection which is ordered*, changeable and do not allow duplicates.As of Python version 3.7, dictionaries are ordered. In Python 3.6 and earlier, dictionaries are unordered.Dictionaries are written with curly brackets, and have keys and values:

Example

Create and print a dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```

Dictionary Items

Dictionary items are ordered, changeable, and does not allow duplicates.

Dictionary items are presented in key:value pairs, and can be referred to by using the key name.

Example

Print the "brand" value of the dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict["brand"])
```

Accessing Items

You can access the items of a dictionary by referring to its key name, inside square brackets:

Example

Get the value of the "model" key:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
x = thisdict["model"]
```

There is also a method called get() that will give you the same result:

Example

Get the value of the "model" key:

```
x = thisdict.get("model")
```

Get Keys

The keys() method will return a list of all the keys in the dictionary.

Example

Get a list of the keys:

```
x = thisdict.keys()
```

The list of the keys is a *view* of the dictionary, meaning that any changes done to the dictionary will be reflected in the keys list.

Example

Add a new item to the original dictionary, and see that the keys list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}

x = car.keys()

print(x) #before the change

car["color"] = "white"

print(x) #after the change
```

Get Values

The values() method will return a list of all the values in the dictionary.

Example

Get a list of the values:

```
x = thisdict.values()
```

The list of the values is a *view* of the dictionary, meaning that any changes done to the dictionary will be reflected in the values list.

Example

Make a change in the original dictionary, and see that the values list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
```

```
}

x = car.values()

print(x) #before the change

car["year"] = 2020

print(x) #after the change
```

Example

Add a new item to the original dictionary, and see that the values list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}

x = car.values()

print(x) #before the change

car["color"] = "red"

print(x) #after the change
```

Get Items

The items() method will return each item in a dictionary, as tuples in a list.

Example

Get a list of the key:value pairs

```
x = thisdict.items()
```

Example

Make a change in the original dictionary, and see that the items list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}

x = car.items()
```

```
print(x) #before the change

car["year"] = 2020

print(x) #after the change
```

Example

Add a new item to the original dictionary, and see that the items list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}

x = car.items()

print(x) #before the change

car["color"] = "red"

print(x) #after the change
```

Check if Key Exists

To determine if a specified key is present in a dictionary use the in keyword:

Example

Check if "model" is present in the dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
if "model" in thisdict:
  print("Yes, 'model' is one of the keys in the thisdict dictionary")
```

Python - Change Dictionary Items

Change Values

You can change the value of a specific item by referring to its key name:

Example

Change the "year" to 2018:

```
thisdict = {
  "brand": "Ford",
```

```
  "model": "Mustang",
  "year": 1964
}
thisdict["year"] = 2018
```

Update Dictionary

The update() method will update the dictionary with the items from the given argument.The argument must be a dictionary, or an iterable object with key:value pairs.

Example

Update the "year" of the car by using the update() method:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.update({"year": 2020})
```

Copy Dictionaries

Copy a Dictionary

You cannot copy a dictionary simply by typing dict2 = dict1, because: dict2 will only be a *reference* to dict1, and changes made in dict1 will automatically also be made in dict2.

There are ways to make a copy, one way is to use the built-in Dictionary method copy().

Example

Make a copy of a dictionary with the copy() method:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
mydict = thisdict.copy()
print(mydict)
```

Another way to make a copy is to use the built-in function dict().

Example

Make a copy of a dictionary with the dict() function:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
```

```
}
mydict = dict(thisdict)
print(mydict)
```

Nested Dictionaries

A dictionary can contain dictionaries, this is called nested dictionaries.

Example

Create a dictionary that contain three dictionaries:

```
myfamily = {
  "child1" : {
    "name" : "Emil",
    "year" : 2004
  },
  "child2" : {
    "name" : "Tobias",
    "year" : 2007
  },
  "child3" : {
    "name" : "Linus",
    "year" : 2011
  }
}
```

Or, if you want to add three dictionaries into a new dictionary:

Example

Create three dictionaries, then create one dictionary that will contain the other three dictionaries:

```
child1 = {
  "name" : "Emil",
  "year" : 2004
}
child2 = {
  "name" : "Tobias",
  "year" : 2007
}
child3 = {
  "name" : "Linus",
  "year" : 2011
}

myfamily = {
```

```
  "child1" : child1,
  "child2" : child2,
  "child3" : child3
}
```

Access Items in Nested Dictionaries

To access items from a nested dictionary, you use the name of the dictionaries, starting with the outer dictionary:

Example

Print the name of child 2:

```
print(myfamily["child2"]["name"])
```

Dictionary Methods

Python has a set of built-in methods that you can use on dictionaries.

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

Learning Outcome 3.3: Understand condition statements

3.3.1 Explaination of Logical condition

Python Conditions and If statements

Python supports the usual logical conditions from mathematics:

- Equals: a == b
- Not Equals: a != b
- Less than: a < b
- Less than or equal to: a <= b
- Greater than: a > b
- Greater than or equal to: a >= b

These conditions can be used in several ways, most commonly in "if statements" and loops.An "if statement" is written by using the if keyword.

Example

If statement:

```
a = 33
b = 200
if b > a:
  print("b is greater than a")
```

Indentation

Python relies on indentation (whitespace at the beginning of a line) to define scope in the code. Other programming languages often use curly-brackets for this purpose.

Example

If statement, without indentation (will raise an error):

```
a = 33
b = 200
if b > a:
print("b is greater than a") # you will get an error
```

Elif

The elif keyword is Python's way of saying "if the previous conditions were not true, then try this condition".

Example

```
a = 33
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
```

In this example a is equal to b, so the first condition is not true, but the elif condition is true, so we print to screen that "a and b are equal".

Else

The else keyword catches anything which isn't caught by the preceding conditions.

Example

```
a = 200
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
```

```
  print("a and b are equal")
else:
  print("a is greater than b")
```

In this example a is greater than b, so the first condition is not true, also the elif condition is not true, so we go to the else condition and print to screen that "a is greater than b".

You can also have an else without the elif:

Example

```
a = 200
b = 33
if b > a:
  print("b is greater than a")
else:
  print("b is not greater than a")
```

Short Hand If

If you have only one statement to execute, you can put it on the same line as the if statement.

Example

One line if statement:

```
if a > b: print("a is greater than b")
```

Short Hand If ... Else

If you have only one statement to execute, one for if, and one for else, you can put it all on the same line:

Example

One line if else statement:

```
a = 2
b = 330
print("A") if a > b else print("B")
```

This technique is known as **Ternary Operators**, or **Conditional Expressions**.

You can also have multiple else statements on the same line:

Example

One line if else statement, with 3 conditions:

```
a = 330
b = 330
print("A") if a >
```

And

The and keyword is a logical operator, and is used to combine conditional statements:

Example

Test if a is greater than b, AND if c is greater than a:

```
a = 200
b = 33
c = 500
if a > b and c > a:
  print("Both conditions are True")
```

Or

The or keyword is a logical operator, and is used to combine conditional statements:

Example

Test if a is greater than b, OR if a is greater than c:

```
a = 200
b = 33
c = 500
if a > b or a > c:
  print("At least one of the conditions is True")
```

Not

The not keyword is a logical operator, and is used to reverse the result of the conditional statement:

Example

Test if a is NOT greater than b:

```
a = 33
b = 200
if not a > b:
  print("a is NOT greater than b")
```

Nested If

You can have if statements inside if statements, this is called *nested* if statements.

Example

```
x = 41

if x > 10:
  print("Above ten,")
  if x > 20:
    print("and also above 20!")
```

```
else:
  print("but not above 20.")
```

The pass Statement

if statements cannot be empty, but if you for some reason have an if statement with no content, put in the pass statement to avoid getting an error.

Example

```
a = 33
b = 200

if b > a:
  pass
```

Learning Outcome 3.4: Identify other functions and classes

3.4.1 Use of Looping

Python For Loops

A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

This is less like the for keyword in other programming languages, and works more like an iterator method as found in other object-orientated programming languages.

With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

Example

Print each fruit in a fruit list:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
```

the break Statement

With the break statement we can stop the loop before it has looped through all the items:

Example

Exit the loop when x is "banana":

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
  if x == "banana":
    break
```

example

Exit the loop when x is "banana", but this time the break comes before the print:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  if x == "banana":
    break
  print(x)
```

The continue Statement

With the continue statement we can stop the current iteration of the loop, and continue with the next:

Example

Do not print banana:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  if x == "banana":
    continue
  print(x)
```

The range() Function

To loop through a set of code a specified number of times, we can use the range() function,

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

Example

Using the range() function:

```
for x in range(6):
  print(x)
```

Note that range(6) is not the values of 0 to 6, but the values 0 to 5.

The range() function defaults to 0 as a starting value, however it is possible to specify the starting value by adding a parameter: range(2, 6), which means values from 2 to 6 (but not including 6):

Example

Using the start parameter:

```
for x in range(2, 6):
  print(x)
```

The range() function defaults to increment the sequence by 1, however it is possible to specify the increment value by adding a third parameter: range(2, 30, **3**):

Example

Increment the sequence with 3 (default is 1):

```
for x in range(2, 30, 3):
  print(x)
```

Else in For Loop

The else keyword in a for loop specifies a block of code to be executed when the loop is finished:

Example

Print all numbers from 0 to 5, and print a message when the loop has ended:

```
for x in range(6):
  print(x)
else:
  print("Finally finished!")
```

Note: The else block will NOT be executed if the loop is stopped by a break statement.

Example

Break the loop when x is 3, and see what happens with the else block:

```
for x in range(6):
  if x == 3: break
  print(x)
else:
  print("Finally finished!")
```

Python Loops

Python has two primitive loop commands:

- while loops
- for loops

The while Loop

With the while loop we can execute a set of statements as long as a condition is true.

Example

Print i as long as i is less than 6:

```
i = 1
while i < 6:
  print(i)
  i += 1
```

The while loop requires relevant variables to be ready, in this example we need to define an indexing variable, i, which we set to 1.

The break Statement

With the break statement we can stop the loop even if the while condition is true:

Example

Exit the loop when i is 3:

```
i = 1
while i < 6:
  print(i)
  if i == 3:
    break
  i += 1
```

3.4.3 Definition of Functions

A function is a block of code which only runs when it is called.You can pass data, known as parameters, into a function.A function can return data as a result.

Creating a Function

In Python a function is defined using the def keyword:

Example

```
def my_function():
  print("Hello from a function")
```

Calling a Function

To call a function, use the function name followed by parenthesis:

Example

```
def my_function():
  print("Hello from a function")

my_function()
```

Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

```
def my_function(fname):
  print(fname + " Refsnes")

my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

Parameters or Arguments?

The terms *parameter* and *argument* can be used for the same thing: information that are passed into a function.
From a function's perspective:
A parameter is the variable listed inside the parentheses in the function definition.
An argument is the value that is sent to the function when it is called.

Number of Arguments

By default, a function must be called with the correct number of arguments. Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less.

Example

This function expects 2 arguments, and gets 2 arguments:

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil", "Refsnes")
```

If you try to call the function with 1 or 3 arguments, you will get an error:

Example

This function expects 2 arguments, but gets only 1:

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil")
```

Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

This way the function will receive a *tuple* of arguments, and can access the items accordingly:

Example

If the number of arguments is unknown, add a * before the parameter name:

```
def my_function(*kids):
  print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

Keyword Arguments

You can also send arguments with the *key = value* syntax.

This way the order of the arguments does not matter.

Example

```
def my_function(child3, child2, child1):
  print("The youngest child is " + child3)

my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus")
```

Arbitrary Keyword Arguments, **kwargs

If you do not know how many keyword arguments that will be passed into your function, add two asterisk: ** before the parameter name in the function definition.

This way the function will receive a *dictionary* of arguments, and can access the items accordingly:

Example

If the number of keyword arguments is unknown, add a double ** before the parameter name:

```
def my_function(**kid):
  print("His last name is " + kid["lname"])

my_function(fname = "Tobias", lname = "Refsnes")
```

Default Parameter Value

The following example shows how to use a default parameter value.

If we call the function without argument, it uses the default value:

Example

```
def my_function(country = "Norway"):
  print("I am from " + country)

my_function("Sweden")
my_function("India")
```

```
my_function()
my_function("Brazil")
```

Passing a List as an Argument

You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches the function:

Example

```
def my_function(food):
  for x in food:
    print(x)

fruits = ["apple", "banana", "cherry"]

my_function(fruits)
```

Return Values

To let a function return a value, use the return statement:

Example

```
def my_function(x):
  return 5 * x

print(my_function(3))
print(my_function(5))
print(my_function(9))
```

The pass Statement

function definitions cannot be empty, but if you for some reason have a function definition with no content, put in the pass statement to avoid getting an error.

Example

```
def myfunction():
  pass
```

Recursion

Python also accepts function recursion, which means a defined function can call itself.

Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of meaning that you can loop through data to reach a result.

The developer should be very careful with recursion as it can be quite easy to slip into writing a function which never terminates, or one that uses excess amounts of memory or processor power. However, when written correctly recursion can be a very efficient and mathematically-elegant approach to programming.

In this example, tri_recursion() is a function that we have defined to call itself ("recurse"). We use the k variable as the data, which decrements (-1) every time we recurse. The recursion ends when the condition is not greater than 0 (i.e. when it is 0).

To a new developer it can take some time to work out how exactly this works, best way to find out is by testing and modifying it.

Example

```
Recursion Example
def tri_recursion(k):
  if(k > 0):
    result = k + tri_recursion(k - 1)
    print(result)
  else:
    result = 0
  return result

print("\n\nRecursion Example Results")
tri_recursion(6)
```

Python Lambda

A lambda function is a small anonymous function.

A lambda function can take any number of arguments, but can only have one expression.

Syntax

lambda *arguments* : *expression*

The expression is executed and the result is returned:

Example

Add 10 to argument a, and return the result:

```
x = lambda a : a + 10
print(x(5))
```

Lambda functions can take any number of arguments:

Example
Multiply argument a with argument b and return the result:

```
x = lambda a, b : a * b
print(x(5, 6))
```

Example

Summarize argument a, b, and c and return the result:

```
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```

Python Arrays
Note: Python does not have built-in support for Arrays, but Python Lists can be used instead.

Arrays

Note: This page shows you how to use LISTS as ARRAYS, however, to work with arrays in Python you will have to import a library, like the NumPy library.

Arrays are used to store multiple values in one single variable:

Note: This page shows you how to use LISTS as ARRAYS, however, to work with arrays in Python you will have to import a library, like the NumPy library.

Arrays are used to store multiple values in one single variable:
Example
Create an array containing car names:

```
cars = ["Ford", "Volvo", "BMW"]
```

What is an Array?

An array is a special variable, which can hold more than one value at a time.

If you have a list of items (a list of car names, for example), storing the cars in single variables could look like this:

```
car1 = "Ford"
car2 = "Volvo"
car3 = "BMW"
```

However, what if you want to loop through the cars and find a specific one? And what if you had not 3 cars, but 300?

The solution is an array!

An array can hold many values under a single name, and you can access the values by referring to an index number.

Access the Elements of an Array

You refer to an array element by referring to the *index number*.

Example

Get the value of the first array item:

```
x = cars[0]
```

Array Methods

Python has a set of built-in methods that you can use on lists/arrays.

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

3.4.3 Definition of Classes/Objects

Python Classes/Objects

Python is an object oriented programming language.Almost everything in Python is an object, with its properties and methods.A Class is like an object constructor, or a "**blueprint**" for creating objects.

Create a Class

To create a class, use the keyword class:

Example

Create a class named MyClass, with a property named x:

```
class MyClass:
  x = 5
```

Create Object

Now we can use the class named MyClass to create objects:

Example

Create an object named p1, and print the value of x:

```
p1 = MyClass()
print(p1.x)
```

The __init__() Function

The examples above are classes and objects in their simplest form, and are not really useful in real life applications.

To understand the meaning of classes we have to understand the built-in __init__() function.

All classes have a function called __init__(), which is always executed when the class is being initiated.

Use the __init__() function to assign values to object properties, or other operations that are necessary to do when the object is being created:

Example

Create a class named Person, use the __init__() function to assign values for name and age:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age
p1 = Person("John", 36)
print(p1.name)
print(p1.age)
```

Note: The __init__() function is called automatically every time the class is being used to create a new object.

The __str__() Function

The __str__() function controls what should be returned when the class object is represented as a string.

If the __str__() function is not set, the string representation of the object is returned:

Example

The string representation of an object WITHOUT the __str__() function:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("John", 36)

print(p1)
```

Object Methods

Objects can also contain methods. Methods in objects are functions that belong to the object.

Let us create a method in the Person class:

Example

Insert a function that prints a greeting, and execute it on the p1 object:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def myfunc(self):
    print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.myfunc()
```

Note: The self-parameter is a reference to the current instance of the class, and is used to access variables that belong to the class.

Object Methods

Objects can also contain methods. Methods in objects are functions that belong to the object.

Let us create a method in the Person class:

Example

Insert a function that prints a greeting, and execute it on the p1 object:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def myfunc(self):
    print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.myfunc()
```

Note: The self parameter is a reference to the current instance of the class, and is used to access variables that belong to the class.

The self-Parameter

The self-parameter is a reference to the current instance of the class, and is used to access variables that belongs to the class.

It does not have to be named self, you can call it whatever you like, but it has to be the first parameter of any function in the class:

Example

Use the words *mysillyobject* and *abc* instead of *self*:

```
class Person:
  def __init__(mysillyobject, name, age):
    mysillyobject.name = name
    mysillyobject.age = age

  def myfunc(abc):
    print("Hello my name is " + abc.name)

p1 = Person("John", 36)
p1.myfunc()
```

Modify Object Properties

You can modify properties on objects like this:

Example
Set the age of p1 to 40:
p1.age = 40

Delete Object Properties

You can delete properties on objects by using the del keyword:
Example
Delete the age property from the p1 object:
del p1.age

The pass Statement

class definitions cannot be empty, but if you for some reason have a class definition with no content, put in the pass statement to avoid getting an error.
Example
```
class Person:
  pass
```

Python Inheritance

Python Inheritance

Inheritance allows us to define a class that inherits all the methods and properties from another class.

Parent class is the class being inherited from, also called base class.

Child class is the class that inherits from another class, also called derived class.

Create a Parent Class

Any class can be a parent class, so the syntax is the same as creating any other class:

Example

Create a class named Person, with firstname and lastname properties, and a printname method:
```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)
```

```
#Use the Person class to create an object, and then execute the printname method:

x = Person("John", "Doe")
x.printname()
```

Create a Child Class

To create a class that inherits the functionality from another class, send the parent class as a parameter when creating the child class:
Example
Create a class named Student, which will inherit the properties and methods from the Person class:

```
class Student(Person):
  pass
```

Note: Use the pass keyword when you do not want to add any other properties or methods to the class.

Now the Student class has the same properties and methods as the Person class

Example

Use the Student class to create an object, and then execute the printname method:

```
x = Student("Mike", "Olsen")
x.printname()
```

Add the __init__() Function

So far we have created a child class that inherits the properties and methods from its parent.

We want to add the __init__() function to the child class (instead of the pass keyword).

Note: The __init__() function is called automatically every time the class is being used to create a new object.

Example

Add the __init__() function to the Student class:

```
class Student(Person):
  def __init__(self, fname, lname):
    #add properties etc.
```

When you add the __init__() function, the child class will no longer inherit the parent's __init__() function.

Note: The child's __init__() function **overrides** the inheritance of the parent's __init__() function.

To keep the inheritance of the parent's __init__() function, add a call to the parent's __init__() function:

Example

```
class Student(Person):
  def __init__(self, fname, lname):
    Person.__init__(self, fname, lname)
```

Now we have successfully added the __init__() function, and kept the inheritance of the parent class, and we are ready to add functionality in the __init__() function.

Use the super() Function

Python also has a super() function that will make the child class inherit all the methods and properties from its parent:

Example

```
class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)
```

By using the super() function, you do not have to use the name of the parent element, it will automatically inherit the methods and properties from its parent.

Add Properties

Example

Add a property called graduationyear to the Student class:

```
class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)
    self.graduationyear = 2019
```

In the example below, the year 2019 should be a variable, and passed into the Student class when creating student objects. To do so, add another parameter in the __init__() function:

Example

Add a year parameter, and pass the correct year when creating objects:

```
class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year
```

```
x = Student("Mike", "Olsen", 2019)
```

Add Methods

Example

Add a method called welcome to the Student class:

```
class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year

  def welcome(self):
    print("Welcome", self.firstname, self.lastname, "to the class of", self.graduationyear)
```

If you add a method in the child class with the same name as a function in the parent class, the inheritance of the parent method will be overridden.

Python Iterators

An iterator is an object that contains a countable number of values.

An iterator is an object that can be iterated upon, meaning that you can traverse through all the values.

Technically, in Python, an iterator is an object which implements the iterator protocol, which consist of the methods __iter__() and __next__().

Iterator vs Iterable

Lists, tuples, dictionaries, and sets are all iterable objects. They are iterable *containers* which you can get an iterator from.

All these objects have a iter() method which is used to get an iterator:

Example

Return an iterator from a tuple, and print each value:

```
mytuple = ("apple", "banana", "cherry")
myit = iter(mytuple)

print(next(myit))
print(next(myit))
print(next(myit))
```

Even strings are iterable objects, and can return an iterator:

Example

Strings are also iterable objects, containing a sequence of characters:

```
mystr = "banana"
myit = iter(mystr)

print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
```

Looping Through an Iterator

We can also use a for loop to iterate through an iterable object:

Example

Iterate the values of a tuple:

```
mytuple = ("apple", "banana", "cherry")

for x in mytuple:
  print(x)
```

Example

Iterate the characters of a string:

```
mystr = "banana"

for x in mystr:
  print(x)
```

The for loop actually creates an iterator object and executes the next() method for each loop.

Example

Create an iterator that returns numbers, starting with 1, and each sequence will increase by one (returning 1,2,3,4,5 etc.):

```
class MyNumbers:
  def __iter__(self):
    self.a = 1
    return self

  def __next__(self):
    x = self.a
```

```
    self.a += 1
    return x

myclass = MyNumbers()
myiter = iter(myclass)

print(next(myiter))
print(next(myiter))
print(next(myiter))
print(next(myiter))
print(next(myiter))
```

Python Polymorphism

The word "polymorphism" means "many forms", and in programming it refers to methods/functions/operators with the same name that can be executed on many objects or classes.

Function Polymorphism

An example of a Python function that can be used on different objects is the len() function.

String

For strings len() returns the number of characters:

Example

```
x = "Hello World!"

print
```

Tuple

For tuples len() returns the number of items in the tuple:

Example

```
mytuple = ("apple", "banana", "cherry")

print(len(mytuple))
```

Dictionary

For dictionaries len() returns the number of key/value pairs in the dictionary:

Example

```
thisdict = {
  "brand": "Ford",
```

```
  "model": "Mustang",
  "year": 1964
}

print(len(thisdict))
```

Class Polymorphism

Polymorphism is often used in Class methods, where we can have multiple classes with the same method name.

For example, say we have three classes: Car, Boat, and Plane, and they all have a method called move():

Example

Different classes with the same method:

```
class Car:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Drive!")

class Boat:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Sail!")

class Plane:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Fly!")

car1 = Car("Ford", "Mustang")       #Create a Car class
```

```
boat1 = Boat("Ibiza", "Touring 20") #Create a Boat class
plane1 = Plane("Boeing", "747")     #Create a Plane class

for x in (car1, boat1, plane1):
  x.move()
```

Look at the for loop at the end. Because of polymorphism we can execute the same method for all three classes.

Inheritance Class Polymorphism

What about classes with child classes with the same name? Can we use polymorphism there?

Yes. If we use the example above and make a parent class called Vehicle, and make Car, Boat, Plane child classes of Vehicle, the child classes inherits the Vehicle methods, but can override them:

Example

Create a class called Vehicle and make Car, Boat, Plane child classes of Vehicle:

```
class Vehicle:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Move!")

class Car(Vehicle):
  pass

class Boat(Vehicle):
  def move(self):
    print("Sail!")

class Plane(Vehicle):
  def move(self):
    print("Fly!")

car1 = Car("Ford", "Mustang") #Create a Car object
boat1 = Boat("Ibiza", "Touring 20") #Create a Boat object
plane1 = Plane("Boeing", "747") #Create a Plane object
```

```
for x in (car1, boat1, plane1):
  print(x.brand)
  print(x.model)
  x.move()
```

Child classes inherits the properties and methods from the parent class.

In the example above you can see that the Car class is empty, but it inherits brand, model, and move() from Vehicle.

The Boat and Plane classes also inherit brand, model, and move() from Vehicle, but they both override the move() method.

Because of polymorphism we can execute the same method for all classes.

3.4.4 Definition of other tools

Python Scope
A variable is only available from inside the region it is created. This is called **scope**.

Local Scope

A variable created inside a function belongs to the ***local scope*** of that function, and can only be used inside that function.

Example

A variable created inside a function is available inside that function:

```
def myfunc():
  x = 300
  print(x)

myfunc()
```

Function Inside Function

As explained in the example above, the variable x is not available outside the function, but it is available for any function inside the function:

Example

The local variable can be accessed from a function within the function:

```
def myfunc():
  x = 300
  def myinnerfunc():
```

```
  print(x)
 myinnerfunc()

myfunc()
```

Global Scope

A variable created in the main body of the Python code is a global variable and belongs to the global scope.

Global variables are available from within any scope, global and local.

Example

A variable created outside of a function is global and can be used by anyone:

```
x = 300

def myfunc():
  print(x)

myfunc()

print(x)
```

Naming Variables

If you operate with the same variable name inside and outside of a function, Python will treat them as two separate variables, one available in the global scope (outside the function) and one available in the local scope (inside the function):

Example

The function will print the local x, and then the code will print the global x:

```
x = 300

def myfunc():
  x = 200
  print(x)

myfunc()

print(x)
```

Global Keyword

If you need to create a global variable, but are stuck in the local scope, you can use the global keyword.

The global keyword makes the variable global.

Example

If you use the global keyword, the variable belongs to the global scope:

```
def myfunc():
  global x
  x = 300

myfunc()

print(x)
```

Also, use the global keyword if you want to make a change to a global variable inside a function.

Example

To change the value of a global variable inside a function, refer to the variable by using the global keyword:

```
x = 300
def myfunc():
  global x
  x = 200
myfunc()
print(x)
```

Python Modules

Consider a module to be the same as a code library.

A file containing a set of functions you want to include in your application.

Create a Module

To create a module just save the code you want in a file with the file extension .py:

Example

Save this code in a file named mymodule.py

```
def greeting(name):
  print("Hello, " + name)
```

Use a Module

Now we can use the module we just created, by using the import statement:

Example

Import the module named mymodule, and call the greeting function:

```
import mymodule

mymodule.greeting("Jonathan")
```

Note: When using a function from a module, use the syntax: module_name.function_name.

Variables in Module

The module can contain functions, as already described, but also variables of all types (arrays, dictionaries, objects etc)

Example

Save this code in the file mymodule.py

```
person1 = {
  "name": "John",
  "age": 36,
  "country": "Norway"
}
```

Example

Import the module named mymodule, and access the person1 dictionary:

```
import mymodule

a = mymodule.person1["age"]
print(a)
```

Naming a Module

You can name the module file whatever you like, but it must have the file extension .py

Re-naming a Module

You can create an alias when you import a module, by using the as keyword:

Example

Create an alias for mymodule called mx:

```
import mymodule as mx

a = mx.person1["age"]
print(a)
```

Built-in Modules

There are several built-in modules in Python, which you can import whenever you like.

Example

Import and use the platform module:

```
import platform

x = platform.system()
print(x)
```

Using the dir() Function

There is a built-in function to list all the function names (or variable names) in a module. The dir() function:

Example

List all the defined names belonging to the platform module:

```
import platform

x = dir(platform)
print(x)
```

Note: The dir() function can be used on *all* modules, also the ones you create yourself.

Import From Module

You can choose to import only parts from a module, by using the from keyword.

Example

The module named mymodule has one function and one dictionary:

```
def greeting(name):
  print("Hello, " + name)

person1 = {
  "name": "John",
  "age": 36,
  "country": "Norway"
}
```

Example

Import only the person1 dictionary from the module:

```
from mymodule import person1

print (person1["age"])
```

Note: When importing using the from keyword, do not use the module name when referring to elements in the module. Example: person1["age"], **not** ~~mymodule.person1["age"].~~

Python Datetime

Python Dates

A date in Python is not a data type of its own, but we can import a module named datetime to work with dates as date objects.

Example

Import the datetime module and display the current date:

```
import datetime

x = datetime.datetime.now()
print(x)
```

Date Output

When we execute the code from the example above the result will be:

```
2023-11-17 01:11:35.259393
```

The date contains year, month, day, hour, minute, second, and microsecond.

The datetime module has many methods to return information about the date object.

Here are a few examples, you will learn more about them later in this chapter:

Example

Return the year and name of weekday:

```
import datetime

x = datetime.datetime.now()

print(x.year)
print(x.strftime("%A"))
```

Creating Date Objects

To create a date, we can use the datetime() class (constructor) of the datetime module.

The datetime() class requires three parameters to create a date: year, month, day.

Example

Create a date object:

```
import datetime

x = datetime.datetime(2020, 5, 17)

print(x)
```

The datetime() class also takes parameters for time and timezone (hour, minute, second, microsecond, tzone), but they are optional, and has a default value of 0, (None for timezone).The datetime object has a method for formatting date objects into readable strings.

The method is called strftime(), and takes one parameter, format, to specify the format of the returned string:

Example

Display the name of the month:

```
import datetime

x = datetime.datetime(2018, 6, 1)

print(x.strftime("%B"))
```

Python JSON

JSON is a syntax for storing and exchanging data.JSON is text, written with JavaScript object notation.

JSON in Python

Python has a built-in package called json, which can be used to work with JSON data.

Example

Import the json module:

```
import json
```

Parse JSON - Convert from JSON to Python

If you have a JSON string, you can parse it by using the json.loads() method.

The result will be a Python dictionary.

Example

Convert from JSON to Python:

```
import json

# some JSON:
x =  '{ "name":"John", "age":30, "city":"New York"}'

# parse x:
y = json.loads(x)

# the result is a Python dictionary:
print(y["age"])
```

Convert from Python to JSON

If you have a Python object, you can convert it into a JSON string by using the json.dumps() method.

Example

Convert from Python to JSON:

```
import json

# a Python object (dict):
x = {
  "name": "John",
  "age": 30,
  "city": "New York"
}

# convert into JSON:
y = json.dumps(x)

# the result is a JSON string:
print(y)
```

You can convert Python objects of the following types, into JSON strings:

- dict
- list
- tuple
- string
- int
- float
- True
- False
- None

Example

Convert Python objects into JSON strings, and print the values:

```
import json

print(json.dumps({"name": "John", "age": 30}))
```

```
print(json.dumps(["apple", "bananas"]))
print(json.dumps(("apple", "bananas")))
print(json.dumps("hello"))
print(json.dumps(42))
print(json.dumps(31.76))
print(json.dumps(True))
print(json.dumps(False))
print(json.dumps(None))
```

When you convert from Python to JSON, Python objects are converted into the JSON (JavaScript) equivalent:

Python	JSON
dict	Object
list	Array
tuple	Array
str	String
int	Number
float	Number
True	true
False	false
None	null

Example

Convert a Python object containing all the legal data types:

```
import json

x = {
  "name": "John",
  "age": 30,
  "married": True,
  "divorced": False,
  "children": ("Ann","Billy"),
  "pets": None,
  "cars": [
    {"model": "BMW 230", "mpg": 27.5},
    {"model": "Ford Edge", "mpg": 24.1}
```

```
  ]
}

print(json.dumps(x))
```

Format the Result

The example above prints a JSON string, but it is not very easy to read, with no indentations and line breaks.

The json.dumps() method has parameters to make it easier to read the result:

Example

Use the indent parameter to define the numbers of indents:

```
json.dumps(x, indent=4)
```

```
import json

x = {
  "name": "John",
  "age": 30,
  "married": True,
  "divorced": False,
  "children": ("Ann","Billy"),
  "pets": None,
  "cars": [
    {"model": "BMW 230", "mpg": 27.5},
    {"model": "Ford Edge", "mpg": 24.1}
  ]
}

# use four indents to make it easier to read the result:
print(json.dumps(x, indent=4))
```

Format the Result

The example above prints a JSON string, but it is not very easy to read, with no indentations and line breaks.

The json.dumps() method has parameters to make it easier to read the result:

Example

Use the indent parameter to define the numbers of indents:

```
json.dumps(x, indent=4)
```

You can also define the separators, default value is (", ", ": "), which means using a comma and a space to separate each object, and a colon and a space to separate keys from values:

Example

Use the separators parameter to change the default separator:

```
json.dumps(x, indent=4, separators=(". ", " = "))
```

Order the Result

The json.dumps() method has parameters to order the keys in the result:

Example

Use the sort_keys parameter to specify if the result should be sorted or not:

```
json.dumps(x, indent=4, sort_keys=True)
```

Python PIP

What is PIP?

PIP is a package manager for Python packages, or modules if you like.

Note: If you have Python version 3.4 or later, PIP is included by default.

What is a Package?

A package contains all the files you need for a module.

Modules are Python code libraries you can include in your project.

Check if PIP is Installed

Navigate your command line to the location of Python's script directory, and type the following:

Example

Check PIP version:

Example

Check PIP version:

C:\Users*Your Name*\AppData\Local\Programs\Python\Python36-32\Scripts>pip --version

Install PIP

If you do not have PIP installed, you can download and install it from this page: https://pypi.org/project/pip/

Download a Package

Downloading a package is very easy.

Open the command line interface and tell PIP to download the package you want.

Navigate your command line to the location of Python's script directory, and type the following:

Example

Download a package named "camelcase":

Python Try Except

The try block lets you test a block of code for errors.

The except block lets you handle the error.

The else block lets you execute code when there is no error.

The finally block lets you execute code, regardless of the result of the try- and except blocks.

Exception Handling

When an error occurs, or exception as we call it, Python will normally stop and generate an error message.

These exceptions can be handled using the try statement:

Example

The try block will generate an exception, because x is not defined:

```
try:
  print(x)
except:
  print("An exception occurred")
```

Since the try block raises an error, the except block will be executed.

Without the try block, the program will crash and raise an error:

Example

This statement will raise an error, because x is not defined:

```
print(x)
```

Many Exceptions

You can define as many exception blocks as you want, e.g. if you want to execute a special block of code for a special kind of error:

Example

Print one message if the try block raises a NameError and another for other errors:

```
try:
  print(x)
except NameError:
  print("Variable x is not defined")
except:
  print("Something else went wrong")
```

User Input

Python allows for user input.

That means we are able to ask the user for input.

The method is a bit different in Python 3.6 than Python 2.7.

Python 3.6 uses the input() method.

Python 2.7 uses the raw_input() method.

The following example asks for the username, and when you entered the username, it gets printed on the screen:

Python 3.6

```
username = input("Enter username:")
print("Username is: " + username)
```

Python 2.7

```
username = raw_input("Enter username:")
print("Username is: " + username)
```

Python stops executing when it comes to the input() function, and continues when the user has given some input.

Python String Formatting

To make sure a string will display as expected, we can format the result with the format() method.

String format()

The format() method allows you to format selected parts of a string.

Sometimes there are parts of a text that you do not control, maybe they come from a database, or user input?

To control such values, add placeholders (curly brackets {}) in the text, and run the values through the format() method:

Example

Add a placeholder where you want to display the price:

```
price = 49
txt = "The price is {} dollars"
print(txt.format(price))
```

You can add parameters inside the curly brackets to specify how to convert the value:

Example

Format the price to be displayed as a number with two decimals:

```
txt = "The price is {:.2f} dollars"
```

EARNING UNIT 4–DEVELOP PYTHON SCRIPT

A Python script is a set of instructions written in the Python programming language that performs a specific task or set of tasks. It is a plain text file containing Python code, usually saved with a .py file extension. Python scripts can range from simple programs to complex applications, depending on the requirements.

Learning Outcome 4.1: Perform File handling

4.1.1 Practice to read file

Python File Open

File handling is an important part of any web application.

Python has several functions for creating, reading, updating, and deleting files.

File Handling

The key function for working with files in Python is the `open()` function.

The `open()` function takes two parameters; *filename*, and *mode*.

There are four different methods (modes) for opening a file:

`"r"` - Read - Default value. Opens a file for reading, error if the file does not exist

`"a"` - Append - Opens a file for appending, creates the file if it does not exist

`"w"` - Write - Opens a file for writing, creates the file if it does not exist

`"x"` - Create - Creates the specified file, returns an error if the file exists

In addition you can specify if the file should be handled as binary or text mode

`"t"` - Text - Default value. Text mode

`"b"` - Binary - Binary mode (e.g. images)

Syntax

To open a file for reading it is enough to specify the name of the file:

```
f = open("demofile.txt")
```

The code above is the same as:

```
f = open("demofile.txt", "rt")
```

Because `"r"` for read, and `"t"` for text are the default values, you do not need to specify them.

Note: Make sure the file exists, or else you will get an error.

Python File Open

Open a File on the Server

Assume we have the following file, located in the same folder as Python:

Assume we have the following file, located in the same folder as Python:

demofile.txt

```
Hello! Welcome to demofile.txt
This file is for testing purposes.
Good Luck!
```

To open the file, use the built-in `open()` function.

The `open()` function returns a file object, which has a `read()` method for reading the content of the file:

Example

```
f = open("demofile.txt", "r")
print(f.read())
```

If the file is located in a different location, you will have to specify the file path, like this:

If the file is located in a different location, you will have to specify the file path, like this:

Example

Open a file on a different location:

```
f = open("D:\\myfiles\welcome.txt", "r")
print(f.read())
```

Read Only Parts of the File

By default the `read()` method returns the whole text, but you can also specify how many characters you want to return:

Example

Return the 5 first characters of the file:

```
f = open("demofile.txt", "r")
print(f.read(5))
```

4.1.2 Python File Write

To write to an existing file, you must add a parameter to the `open()` function:

`"a"` - Append - will append to the end of the file

`"w"` - Write - will overwrite any existing content

Example

Open the file "demofile2.txt" and append content to the file:

```
f = open("demofile2.txt", "a")
f.write("Now the file has more content!")
f.close()

#open and read the file after the appending:
f = open("demofile2.txt", "r")
print(f.read())
```

Example

Open the file "demofile3.txt" and overwrite the content:

```
f = open("demofile3.txt", "w")
f.write("Woops! I have deleted the content!")
f.close()

#open and read the file after the overwriting:
f = open("demofile3.txt", "r")
print(f.read())
```

Note: the "w" method will overwrite the entire file.

Create a New File

To create a new file in Python, use the `open()` method, with one of the following parameters:

"x" - Create - will create a file, returns an error if the file exist

"a" - Append - will create a file if the specified file does not exist

"w" - Write - will create a file if the specified file does not exist

Example

Create a file called "myfile.txt":

```
f = open("myfile.txt", "x")
```

Result: a new empty file is created!

Example

Create a new file if it does not exist:

```
f = open("myfile.txt", "w")
```

4.1.3 Python Delete File

Delete a File

To delete a file, you must import the OS module, and run its `os.remove()` function:

Example

Remove the file "demofile.txt":

```
import os
os.remove("demofile.txt")
```

Check if File exist:

To avoid getting an error, you might want to check if the file exists before you try to delete it:

Example

Check if file exists, *then* delete it:

```
import os
if os.path.exists("demofile.txt"):
  os.remove("demofile.txt")
else:
  print("The file does not exist")
```

Delete Folder

To delete an entire folder, use the `os.rmdir()` method:

Example

Remove the folder "myfolder":

```
import os
os.rmdir("myfolder")
```

Note: You can only remove *empty* folders.

Learning Outcome 4.2: Determine Python library

A Python library is a collection of modules or packages that provide pre-written code and functionalities, allowing developers to perform specific tasks without having to write code from scratch. Libraries in Python are designed to be reusable and serve as building blocks for applications. They can include functions, classes, and constants that help programmers perform common operations efficiently.

NumPy is a Python library.

NumPy is used for working with arrays.

NumPy is short for "Numerical Python".

you can use the NumPy module, and modify the code to see the result.

Example

Create a NumPy array:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)

print(type(arr))
```

Part 2 Fundamentals of AI

1.1 Introduction to machine learning
1.1.1 Machine learning overview

Definition

Machine Learning (ML) is a subfield of artificial intelligence (AI) that focuses on the development of algorithms and statistical models that enable computer systems to perform a task without explicit programming. The primary goal of machine learning is to allow computers to **learn from data and impro**ve their performance over time.

In traditional programming, humans explicitly write code to instruct a computer on how to perform a specific task. In contrast, machine learning algorithms learn **patterns and relationships from data, enabling them to make predictions or decisions** without being explicitly programmed for each scenario.

Machine learning life cycle

The machine learning lifecycle encompasses the entire process of developing, deploying, and maintaining a machine learning model. The general overview of the machine learning lifecycle:

1. **Define the Problem**

Clearly articulate the problem you want to solve and determine if machine learning is the appropriate solution.

2. **Collect and Prepare Data**

Gather relevant data that will be used **to train, validate, and test the** machine learning model.

Clean and preprocess the data, handling missing values, outliers, and ensuring data is in a suitable format.

3. **Exploratory Data Analysis (EDA)**

Understand the characteristics of the data through visualization and statistical analysis.

Identify patterns, correlations, and potential features for the model.

4. **Feature Engineering**

Select or create features that are most relevant to the problem.

Transform and scale features as needed for model training.

5. **Select a Model**

Choose a machine learning algorithm or a combination of algorithms based on the nature of the problem and the characteristics of the data.

Split the Data:

Divide the dataset into training, validation, and test sets to assess the model's performance accurately.

6. **Train the Model**

Feed the training data into the selected model and adjust its parameters to learn patterns from the data.

Validate the model on the validation set to fine-tune hyperparameters and prevent overfitting.

7. **Evaluate the Model**

Assess the model's performance on the test set to ensure it generalizes well to new, unseen data.

Metrics such as accuracy, precision, recall, and F1 score are commonly used for evaluation.

8. **Deploy the Model**

If the model meets the desired performance criteria, deploy it to a production environment where it can make predictions on new data.

1.1.2 Applications

Machine learning has found applications in a wide range of fields, transforming industries and improving various processes.

Image and Speech Recognition

Machine learning is used in image recognition for tasks such as facial recognition, object detection, and image classification. Similarly, it is applied in speech recognition for converting spoken language into text.

Natural Language Processing (NLP)

NLP involves the interaction between computers and human language. Machine learning is applied in tasks such as sentiment analysis, language translation, chatbots, and text summarization.

Healthcare

Machine learning is employed for predictive analytics, disease identification, personalized treatment plans, drug discovery, and medical image analysis, improving diagnostics and patient care.

Finance:

In finance, machine learning is used for credit scoring, fraud detection, algorithmic trading, portfolio management, and risk assessment. These applications help optimize financial processes and enhance decision-making.

Recommendation Systems

E-commerce platforms, streaming services, and content providers use machine learning to create recommendation systems that suggest products, movies, or content based on user preferences and behavior.

Autonomous Vehicles

Machine learning plays a crucial role in developing self-driving cars. It enables vehicles to perceive their surroundings, make decisions, and navigate safely through various environments.

1.1.3 Advantages and disadvantages

1 Easily identifies trends and patterns

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

2 No human intervention needed (automation)

With ML, you don’t need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on

their own. A common example of this is anti-virus softwares; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

3 Handling multi-dimensional and multi-variety data

Machine Learning algorithms are good at handling data that are multi-dimensional and multi-variety, and they can do this in d

Disadvantages of Machine Learning

With all those advantages to its powerfulness and popularity, Machine Learning isn't perfect. The following factors serve to limit it

1. **Data Acquisition**

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

2. **Time and Resources**

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

3. **Interpretation of Results**

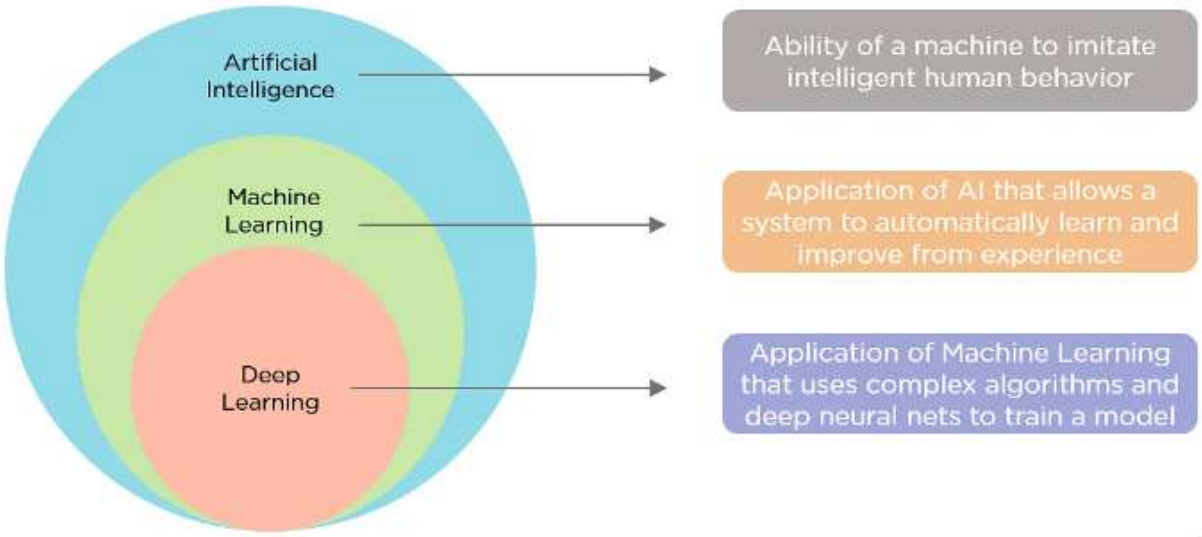
Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

4. **High error-susceptibility**

ML is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

1.1.4 Difference between machine learning, artificial intelligence, and deep learning

Here is an illustration designed to help us understand the fundamental differences between artificial intelligence, machine learning, and deep learning.



Artificial intelligence is the overarching system. Machine learning is a subset of AI. Deep learning is a subfield of machine learning, and neural networks make up the backbone of deep learning algorithms.

Artificial Intelligence, deep learning, machine learning—whatever you're doing if you don't understand it—learn it. Because otherwise, you're going to be a dinosaur within 3 years.

The three terms are often used interchangeably, but they do not quite refer to the same things.

Here is an illustration designed to help us understand the fundamental differences between artificial intelligence, machine learning, and deep learning.

Artificial Intelligent, the broadest term of the three, is used to classify machines that mimic human intelligence and human cognitive functions like problem-solving and learning. AI uses predictions and automation to optimize and solve complex tasks that humans have historically done, such as facial and speech recognition, decision making and translation.

Machine learning is a subset of artificial intelligence that allows for optimization. When set up correctly, it helps you make predictions that minimize the errors that arise from merely guessing. For example, companies like Amazon use machine learning to recommend products to a specific customer based on what they've looked at and bought before.

As our article on **deep learning** explains, deep learning is a subset of machine learning. The primary difference between machine learning and deep learning is how each algorithm learns and how much data each type of algorithm uses.

Neural networks, also called artificial neural networks (ANNs) or simulated neural networks (SNNs), are a subset of machine learning and are the backbone of **deep learning algorithms**. They are called “neural” because they mimic how neurons in the brain signal one another.

1.1.2 Types of Machine Learning

- **Supervised Learning**

Supervised learning uses **labeled data** (data with known answers) to train algorithms to:

- Classify Data
- Predict Outcomes

Supervised learning can **classify** data like "What is spam in an e-mail", based on known spam examples.

Supervised learning can **predict** outcomes like predicting what kind of video you like, based on the videos you have played.

- **Unsupervised Learning**

In this type, the algorithm is given unlabeled data and must find patterns or relationships within the data on its own. Clustering and dimensionality reduction are common tasks in unsupervised learning.

Unsupervised learning is used to predict undefined relationships like meaningful patterns in data.

It is about creating computer algorithms than can improve themselves.

It is expected that machine learning will shift to unsupervised learning to allow programmers to solve problems without creating models.

- **Reinforcement Learning**

Involves training a model to make sequences of decisions by rewarding or penalizing the model based on the outcomes of its actions. The model learns to maximize cumulative rewards over time.

Reinforcement learning is based on non-supervised learning but receives feedback from the user whether the decisions is good or bad. The feedback contributes to improving the model.

1.1.3 **Programming languages and tools**

Programming languages involved in **Machine Learning** and Artificial Intelligence are:

- LISP
- R
- Python
- C++
- Java
- JavaScript
-

LISP

LISP is the second oldest programming language in the world (1958), one year younger than Fortran (1957).

The term **Artificial Intelligence** was made up by **John McCarthy** who invented LISP.

LISP was founded on the theory of **Recursive Functions** (a function appears in its own definition).

Recursive Functions can be written as self-modifying functions, and this is very suitable for Machine Learning programs where "self-learning" is an important part of the program

The R Language

R is a programming language for **Graphics** and **Statistical** computing.

R comes with a wide set of statistical and graphical techniques for:

- Linear Modeling
- Nonlinear Modeling
- Statistical Tests
- Time-series Analysis
- Classification
- Clustering

Python

Python is a general-purpose coding language. It can be used for all types of programming and software development.

Python is typically used for server development, like building web apps for web servers.

Python is also typically used in **Data Science**.

An advantage for using Python is that it comes with some very suitable libraries:

- NumPy (Library for working with Arrays)
- SciPy (Library for Statistical Science)
- Matplotlib (Graph Plotting Library)
- NLTK (Natural Language Toolkit)
- TensorFlow (Machine Learning)

C++

C++ holds the title: "**The worlds fastest programming language**".

Because of the speed, C++ is a preferred language when programming Computer Games.

It provides faster execution and has less response time which is applied in search engines and development of computer games.

Google uses C++ in Artificial Intelligence and Machine Learning programs for SEO (Search Engine Optimization).

SHARK is a super-fast C++ library with support for supervised learning algorithms, linear regression, neural networks, and clustering.

MLPACK is also a super-fast machine learning library for C++.

Java

Java is another general-purpose coding language that can be used for all types of software development.

For Machine Learning, Java is mostly used to create algorithms, and neural networks.

1.2 Setting up machine learning environment

The choice of tool depends on the specific requirements of the project and the preferences of the user or development team.

There are numerous machine learning tools and frameworks available, catering to different needs and preferences. Here are some popular ones:

Scikit-Learn: Scikit-Learn is an open-source machine learning library for Python. It provides simple and efficient tools for data analysis and modeling, including various algorithms for classification, regression, clustering, and more.

TensorFlow:Description: Developed by Google, TensorFlow is an open-source machine learning framework widely used for building and training deep learning models. It supports both neural networks and traditional machine learning algorithms.

PyTorch: PyTorch is an open-source deep learning framework developed by Facebook. It is known for its dynamic computation graph, making it flexible and suitable for research and experimentation in addition to production use.

Keras:Description: Keras is a high-level neural networks API that can run on top of **TensorFlow, Theano, or Microsoft Cognitive Toolkit**. It simplifies the process of building and experimenting with deep learning models.

Scikit-Image:Description: This is an image processing library built on top of Scikit-Learn. It provides algorithms for segmentation, feature extraction, and other image processing tasks.

Pandas:Description: While not a machine learning library per se, Pandas is a powerful Python library for data manipulation and analysis. It is often used for preparing and cleaning data before feeding it into machine learning models.

NLTK (Natural Language Toolkit): NLTK is a library for working with human language data. It provides easy-to-use interfaces to work with linguistic data for tasks such as classification, tokenization, stemming, tagging, parsing, and more.

RapidMiner: RapidMiner is an integrated data science platform that provides tools for data preparation, machine learning, and model deployment. It offers a visual interface for designing and executing machine learning workflows.

Microsoft Azure Machine Learning:Azure ML is a cloud-based service that provides tools for building, deploying, and managing machine learning models on the Microsoft Azure cloud. It supports various frameworks, including TensorFlow and PyTorch.

Google Colab:Google Colab is a free, cloud-based platform that provides a Jupyter notebook environment with free GPU and TPU support. It is often used for collaborative machine learning projects and experiments.

1.3 Data Collection and Acquisition

data collection is in fact the first and most fundamental step in the machine learning pipeline. It's part of the complex data processing phase within an ML lifecycle. From this comes another important point: data collection directly impacts the performance of an ML model and the final results.

Data acquisition is the process of taking measurements of real-world physical occurrences using signals and digitizing them so that a computer and software may alter them.

t is the procedure of locating relevant business data, formatting the information into the necessary business form, and loading the data into the specified system. he three fundamental parts of every data acquisition system are a sensor, signal conditioning, and an analog-to-digital converter (ADC).

1.3.2 Explanation of data, big data and ML dataset

With Machine Learning, **data** is collections of facts:

Type	Examples
Numbers	Prices. Dates.
Measurements	Size. Height. Weight.
Words	Names and Places.
Observations	Counting Cars.
Descriptions	It is cold.

Big Data

Big data is data that is impossible for humans to process without the assistance of advanced machines.

Big data does not have any definition in terms of size, but datasets are becoming larger and larger as we continously collect more and more data and store data at a lower and lower cost.

Data Set

In the mind of a computer, a data set is any collection of data. It can be anything from an array to a complete database.

Machine learning datasets are important for two reasons: they allow you **to train** your machine learning models, and they provide a benchmark **for measuring the accuracy of** your models.

1.3.3 Key Characteristics for ML dataset

high-quality machine learning dataset should possess accurate and reliable data (Quality), an adequate number of instances (Quantity), and sufficient diversity to cover various scenarios (Variability).

Quality

Quality refers to the reliability and accuracy of the data within the dataset. High-quality data is free from errors, inconsistencies, and inaccuracies.

High-quality data contributes to **accurate and reliable predictions** by ensuring that the model learns meaningful patterns from the data.

Reliable data enhances the **trustworthiness** of the machine learning model, making it more suitable for decision-making in real-world scenarios.

Quality data **reduces the likelihood of biases in the** dataset, which can lead to biased model predictions.

Quantity

Quantity refers to the **size of the dataset**, measured in terms of the number of instances or data points. A larger dataset generally provides more information for training a robust model.

Larger datasets often lead to **better generalization**, allowing the model to perform well on **new, unseen data.**

Adequate data helps **prevent overfitting, where** the model **memorizes** the training data instead of learning general patterns.

A larger dataset provides more statistically **significant insights and patterns,** leading to more reliable conclusions.

Variability

Variability refers to the diversity and range of the data within the dataset. A dataset with sufficient variability covers different scenarios, conditions, and potential outcomes.

Importance:

A dataset with variability helps **train a robust** model that can **handle diverse situations** and variations present in real-world data.

Avoiding Overfitting: Including diverse examples reduces the risk of overfitting to specific patterns present in the training data but not applicable to new instances.

Variability contributes to better generalization, allowing the model to make accurate predictions across different contexts

1.3.4 Types of datasets

A machine learning dataset is a collection of data that is used to train the model. A dataset acts as an example to teach the machine learning algorithm how to make predictions. The common types of data include.

- Text data
- Image data
- Audio data
- Video data
- Numeric data

Source of Dataset

Sourcing a dataset depends on the requirements and scope of the project. If your project does not require highly personalized data, some common datasets that can be sourced from vendors for different types of projects.

1. Natural language processing datasets

Datasets are used for, text analytics, and language translation. These types of datasets are large in size and require heavy computational power.

Some popular NLP datasets include:

Amazon Reviews

The Big ad NLP Database

Wikipedia Links Data

2. **Open datasets**

These ready-to-use datasets are freely available online for anyone to download, modify, and distribute without legal or financial restrictions. These datasets are regularly updated and are compatible with most ML frameworks. The only drawback is that open datasets lack personalization.

Popular open datasets include:

Google dataset search

AWS public datasets

Kaggle datasets

3. **Public government datasets**

These datasets are used for government projects that are implemented for the public. For example, these datasets can include a certain population's census or demographic data. These datasets can be used to make policies or train AI/ML models for immigration decision-making, chatbots that answer citizen queries, city infrastructure planning systems, etc.

Popular public government datasets include:

- Data.Gov.uk
- EU open data portal
- Data USA

4. **Image datasets**

Image datasets include both image and video data. This type of dataset is used to train computer vision systems for facial recognition, autonomous vehicle systems, retail security systems, etc. These datasets required high-quality image annotation to be used.

Popular image datasets include:

- Google's open images
- Coco dataset
- Imagenet
- Baidu ApolloScape Dataset
- Waymo Open Dataset

5. **Audio datasets**

These datasets are used to train AI/ML models for voice recognition, music recognition, etc.

Popular audio datasets include:

- Environmental audio datasets
- Speech commands dataset
- Free music archive (FMA)
- Flickr audio caption corpus

Common voice

6. **Healthcare Datasets**

These datasets are used to train medical imaging systems or medical diagnosis systems. They are usually large in size and require heavy computational and high-quality medical annotation.

Popular healthcare datasets include:

MIMIC Critical Care Database

Healthdata.gov

You can also check out our data-driven list of data collection/harvesting services to find the option that best suits your project needs.

7. **Generative AI**

Generative AI, particularly for models like Generative Adversarial Networks (GANs), has transformed the landscape of data creation and augmentation. Creating datasets using generative AI addresses several challenges in machine learning.

n situations where collecting real-world data is expensive, time-consuming, or ethically challenging, generative models can supplement or even replace traditional data collection methods.

For instance, medical imaging datasets can be augmented using GANs to generate more samples of rare conditions, making it easier to train models that can detect and diagnose these conditions.

Additionally, in domains like computer vision (CV), generating diverse data helps in mitigating overfitting and improving the robustness of the trained models. This synthesized data, when used judiciously alongside real data, can help to train more effective and accurate machine learning models, while saving resources and time in the data collection phase.

1.4 Data Preprocessing

Data Processing is the task of converting data from a given form to a much more usable and desired form i.e. making it more meaningful and informative. Using Machine Learning algorithms, mathematical modeling, and statistical knowledge, this entire process can be automated. The output of this complete process can be in any desired form like graphs, videos, charts, tables, images, and many more, depending on the task we are performing and the requirements of the machine.

The **goal** of data processing is to clean, transform, and prepare the data in a format that is suitable for modeling.

1. **The main steps involved in data processing typically include:**

- **Data collection**: This is the process of gathering data from various sources, such as sensors, databases, or other systems. The data may be structured or unstructured, and may come in various formats such as text, images, or audio.
- **Data preprocessing:** This step involves cleaning, filtering, and transforming the data to make it suitable for further analysis. This may include removing **missing values, scaling or normalizing the data, or converting it to a different format.**
- **Data analysis**: In this step, the data is analyzed using various techniques such as statistical analysis, machine learning algorithms, or data visualization. The goal of this step is to derive insights or knowledge from the data.
- **Data interpretation**: This step involves interpreting the results of the data analysis and drawing conclusions based on the insights gained. It may also involve presenting the findings in a clear and concise manner, such as through reports, dashboards, or other visualizations.
- **Data storage and management**: Once the data has been processed and analyzed, it must be stored and managed in a way that is secure and easily accessible. This may involve storing the data in a database, cloud storage, or other systems, and implementing backup and recovery strategies to protect against data loss.
- **Data visualization and reporting: Finally, the results of the data analysis are presented to stakeholders in a** format that is easily understandable and actionable. This may involve creating visualizations, reports, or dashboards that highlight key findings and trends in the data.

2. **Characteristics of quality Data**

Accuracy refers to the correctness and precision of the data. Accurate data is free from errors and reflects the true values or characteristics it is intended to represent.

Completeness measures the extent to which the dataset includes all the necessary information. A complete dataset contains all the expected records and fields without missing values.

Consistency refers to the uniformity and standardization of data across the dataset. Consistent data maintains the same format, units, and conventions throughout.

Relevance assesses the significance of the data to the goals and objectives of the analysis or modeling. Relevant data is directly related to the problem at hand.

Validity refers to the extent to which the data accurately represents the real-world entities or phenomena it is supposed to capture. Valid data is meaningful and appropriate for its intended use.

3. **Data cleaning for inconsistencies rectification**

Data cleaning is one of the important parts of machine learning. It plays a significant part in building a model. However, the success or failure of a project relies on proper data cleaning. Professional data scientists usually invest a very large portion of their time in this step because of the belief that “Better data beats fancier algorithms”.

Data cleaning, also known as data cleansing or data preprocessing, is a crucial step in the data science pipeline that involves identifying and correcting or removing errors, inconsistencies, and

Machine Learing model

Linear Regression

Linear Regression is a supervised learning algorithm applied when the target or dependent variable is a continuous real number. It establishes a relationship between the dependent variable y and one or more independent variables x using a best-fit line. The algorithm operates based on the principle of Ordinary Least Squares (OLS) or Mean Square Error (MSE). In statistics, OLS is a method for estimating unknown parameters of the linear regression function, with its objective being to minimize the sum of square differences between the observed dependent variable in the given dataset and those predicted by the linear regression function.

We will use x_i to represent the independent variable and y_i to represent the dependent variable. A pair (x_i, y_i) is referred to as a training example, and the subscript i in the notation serves as an index into the training set. If there are m training examples, then $i = 1, 2, 3, ..., m$.

The aim of supervised learning is to learn a hypothesis function h, given a training set, that can be used to estimate y based on x. The hypothesis function is represented as:

$$h_\theta(x_i) = \theta_0 + \theta_1 x_i$$

Here, θ_0 and θ_1 are the parameters of the hypothesis, and this equation corresponds to Simple/Univariate Linear Regression.

The aim of supervised learning is to learn a hypothesis function h, given a training set, that can be used to estimate y based on x. The hypothesis function is represented as:

$$h_\theta(x_i) = \theta_0 + \theta_1 x_i$$

Here, θ_0 and θ_1 are the parameters of the hypothesis, and this equation corresponds to Simple/Univariate Linear Regression.

For Multiple Linear Regression, where more than one independent variable exists, x_{ij} is used to denote the j-th independent variable, and y_i represents the dependent variable. If there are n independent variables, then $j = 1, 2, 3, \ldots, n$. The hypothesis function is represented as:

$$h_\theta(x_i) = \theta_0 + \theta_1 x_{i1} + \theta_2 x_{i2} + \ldots + \theta_j x_{ij} + \ldots + \theta_n x_{in}$$

Here, $\theta_0, \theta_1, \ldots, \theta_j, \ldots, \theta_n$ are the parameters of the hypothesis, m is the number of training examples, n is the number of independent variables, and x_{ij} is the i-th training example of the j-th feature.

Import Library and Dataset

Now we will import couple of python library required for our analysis and import dataset

```
import pandas  as pd #Data manipulation
import numpy as np #Data manipulation
import matplotlib.pyplot as plt # Visualization
import seaborn as sns  #Visualization
# Set the default figure size for Matplotlib plots to [8, 5] inches
plt.rcParams['figure.figsize'] = [8, 5]
# Set the default font size for text elements in the plot to 14 points
plt.rcParams['font.size'] = 14
# Set the default font weight to bold for text elements in the plot
plt.rcParams['font.weight'] = 'bold'
```

```
# Apply the 'seaborn-whitegrid' style to the plot for a white background with a grid
plt.style.use('seaborn-whitegrid')
```

rcParams (short for "runtime configuration parameters") is a configuration system used in Matplotlib, a popular Python plotting library. It allows you to customize the behavior and appearance of Matplotlib plots at runtime by modifying a dictionary-like object. rcParams contains key-value pairs where each key corresponds to a configuration parameter, and the associated value is the setting for that parameter. By modifying these parameters, you can control various aspects of the plot, such as figure size, font size, colors, and more. Here are some common examples of configuration parameters that can be set using rcParams:

figure.figsize: Sets the default figure size for plots.

font.size: Sets the default font size for text elements in the plot.

font.weight: Sets the default font weight for text elements.

axes.grid: Specifies whether the grid should be displayed on axes.

```
df=pd.read_csv("C:/Users/user/Downloads/insurance.csv")
```

Now we have import dataset. When we look at the shape of dataset it has return as (1338,7).So there are m=1338 training exaple and =7 independent variable. The target variable here is charges and remaining six variables such as age, sex, bmi, children, smoker, region are independent variable. There are multiple independent variable, so we need to fit Multiple linear regression. Then the hypothesis function looks like

$$\mathbf{h_\theta(x_i) = \theta_0 + \theta_1\,age + \theta_2\,sex + \theta_3\,bmi + \theta_4\,children + \theta_5\,smoker + \theta_6\,region}$$

This multiple linear regression equation for given dataset.

If $\mathbf{i = 1}$ then

$$\mathbf{h_\theta(x_1) = \theta_0 + \theta_1\,19 + \theta_2\,female + \theta_3\,27.900 + \theta_4\,1 + \theta_5\,yes + \theta_6\,southwest}$$
$$\mathbf{y_1 = 16884.92400}$$

If $\mathbf{i = 3}$ then

$$\mathbf{h_\theta(x_3) = \theta_0 + \theta_1\,28 + \theta_2\,male + \theta_3\,33.000 + \theta_4\,3 + \theta_5\,no + \theta_6\,northwest}$$
$$\mathbf{y_3 = 4449.46200}$$

Note: In python index starts from 0.

$$\mathbf{x_1} = (\,x_{11} \quad x_{12} \quad x_{13} \quad x_{14} \quad x_{15} \quad x_{16}\,)$$
$$= (\,19 \quad female \quad 27.900 \quad 1 \quad no \quad northwest\,)$$

Matrix Formulation

In general we can write above vector as

$$\mathbf{x_{ij}} = (\,\mathbf{x_{i1}}\ \mathbf{x_{i2}}\ \ldots\ \mathbf{x_{in}}\,)$$

Now we combine all aviable individual vector into single input matrix of size (m, n) and denoted it by $\mathbf{X}$ input matrix, which consist of all training exaples,

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \ldots & x_{1n} \\ x_{21} & x_{22} & \ldots & x_{2n} \\ x_{31} & x_{32} & \ldots & x_{3n} \\ \vdots & \vdots & \vdots & \vdots \\ x_{m1} & x_{m2} & \ldots & x_{mn} \end{pmatrix}_{(m,n)}$$

We represent parameter of function and dependent variable in vactor form as

$$\theta = \begin{pmatrix} \theta_0 \\ \theta_1 \\ . \\ . \\ \theta_j \\ . \\ . \\ \theta_n \end{pmatrix}_{(n+1,1)} \qquad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ . \\ . \\ y_i \\ . \\ . \\ y_m \end{pmatrix}_{(m,1)}$$

So we represent hypothesis function in vectorize form

$$\mathbf{h}_\theta(\mathbf{x}) = \mathbf{X}\theta$$

""" for our visualization purpose will fit line using seaborn library only for bmi as independent v ariable
and charges as dependent variable"""

```
# Apply the 'seaborn-whitegrid' style to the plot for a white background with a grid
plt.style.use('seaborn-whitegrid')
# Create a linear regression plot using seaborn's lmplot
sns.lmplot(x='bmi', y='charges', data=df, aspect=2, height=6)
# Set the x-axis label
plt.xlabel('Body Mass Index $(kg/m^2)$: as Independent variable')
# Set the y-axis label
plt.ylabel('Insurance Charges: as Dependent variable')
# Set the title of the plot
plt.title('Charge Vs BMI')
```

sns.lmplot is a function provided by the Seaborn library, which is built on top of Matplotlib and is specifically designed for statistical data visualization. The **lmplot** function is used for plotting

linear models and is particularly handy for visualizing the relationship between two variables along with a regression line.

Cost function

A cost function measures how much error in the model is in terms of ability to estimate the relationship between x and y. We can measure the accuracy of our hypothesis function by using a cost function. This takes an average difference of observed dependent variable in the given the dataset and those predicted by the hypothesis function.

$$\mathbf{J}(\theta) = \frac{\mathbf{1}}{\mathbf{m}} \sum_{\mathbf{i=1}}^{\mathbf{m}} (\hat{\mathbf{y}}_\mathbf{i} - \mathbf{y}_\mathbf{i})^\mathbf{2}$$

$$\mathbf{J}(\theta) = \frac{\mathbf{1}}{\mathbf{m}} \sum_{\mathbf{i=1}}^{\mathbf{m}} (\mathbf{h}_\theta(\mathbf{x}_\mathbf{i}) - \mathbf{y}_\mathbf{i})^\mathbf{2}$$

To implement the linear regression, take training example add an extra column that is x_0 feature, where $\mathbf{x_0} = \mathbf{1}$. $\mathbf{x_o} = (\, x_{i0}\ x_{i1}\ x_{i2}\ \ldots\ x_{mi}\,)$,where $\mathbf{x_{i0}} = \mathbf{0}$ and input matrix will become as

$$\mathbf{X} = \begin{pmatrix} x_{10} & x_{11} & x_{12} & \ldots & x_{1n} \\ x_{20} & x_{21} & x_{22} & \ldots & x_{2n} \\ x_{30} & x_{31} & x_{32} & \ldots & x_{3n} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{m0} & x_{m1} & x_{m2} & \ldots & x_{mn} \end{pmatrix}_{(m,n+1)}$$

Each of the m input samples is similarly a column vector with n+1 rows x_0 being 1 for our convenience, that is $\mathbf{x_{10}, x_{20}, x_{30} \ldots . x_{m0}} = \mathbf{1}$. Now we rewrite the ordinary least square cost function in matrix form as

$$\mathbf{J}(\theta) = \frac{\mathbf{1}}{\mathbf{m}} (\mathbf{X}\theta - \mathbf{y})^\mathbf{T} (\mathbf{X}\theta - \mathbf{y})$$

Let's look at the matrix multiplication concept,the multiplication of two matrix happens only if number of column of firt matrix is equal to number of row of second matrix. Here input matrix $\mathbf{X}$ of size $(\mathbf{m}, \mathbf{n+1})$, parameter of function is of size $(n+1, 1)$ and dependent variable vector of size $(\mathbf{m}, \mathbf{1})$. The product of matrix $\mathbf{X_{(m,n+1)}} \theta_{\mathbf{(n+1,1)}}$ will return a vector of size $(\mathbf{m}, \mathbf{1})$, then product of $(\mathbf{X}\theta - \mathbf{y})^\mathbf{T}_{\mathbf{(1,m)}} (\mathbf{X}\theta - \mathbf{y})_{\mathbf{(m,1)}}$ will return size of unit vector.

Normal Equation

The normal equation is an analytical solution to the linear regression problem with a ordinary least square cost function. To minimize our cost function, take partial derivative of $\mathbf{J}(\theta)$ with respect to θ and equate to 0. The derivative of function is nothing but if a small change in input what would be the change in output of function.

$$\mathbf{min}_{\theta_0,\theta_1..\theta_n}\mathbf{J}(\theta_0,\theta_1..\theta_n)$$
$$\frac{\partial \mathbf{J}(\theta_j)}{\partial \theta_j} = 0$$

where $\mathbf{j} = 0, 1, 2, \ldots . \mathbf{n}$

Now we will apply partial derivative of our cost function,

$$\frac{\partial \mathbf{J}(\theta_j)}{\partial \theta_j} = \frac{\partial}{\partial \theta}\frac{1}{\mathbf{m}}(\mathbf{X}\theta - \mathbf{y})^{\mathbf{T}}(\mathbf{X}\theta - \mathbf{y})$$

I will throw $\frac{1}{\mathbf{m}}$ part away since we are going to compare a derivative to 0. And solve $\mathbf{J}(\theta)$,

$$\mathbf{J}(\theta) = (\mathbf{X}\theta - \mathbf{y})^{\mathbf{T}}(\mathbf{X}\theta - \mathbf{y})$$
$$= (\mathbf{X}\theta)^{\mathbf{T}} - \mathbf{y}^{\mathbf{T}})(\mathbf{X}\theta - \mathbf{y})$$

$$= (\theta^{\mathbf{T}}\mathbf{X}^{\mathbf{T}} - \mathbf{y}^{\mathbf{T}})(\mathbf{X}\theta - \mathbf{y})$$
$$= \theta^{\mathbf{T}}\mathbf{X}^{\mathbf{T}}\mathbf{X}\theta - \mathbf{y}^{\mathbf{T}}\mathbf{X}\theta - \theta^{\mathbf{T}}\mathbf{X}^{\mathbf{T}}\mathbf{y} + \mathbf{y}^{\mathbf{T}}\mathbf{y}$$
$$= \theta^{\mathbf{T}}\mathbf{X}^{\mathbf{T}}\mathbf{X}\theta - 2\theta^{\mathbf{T}}\mathbf{X}^{\mathbf{T}}\mathbf{y} + \mathbf{y}^{\mathbf{T}}\mathbf{y}$$

Here $\mathbf{y}^{\mathbf{T}}_{(\mathbf{1},\mathbf{m})}\mathbf{X}_{(\mathbf{m},\mathbf{n}+\mathbf{1})}\theta_{(\mathbf{n}+\mathbf{1},\mathbf{1})} = \theta^{\mathbf{T}}_{(\mathbf{1},\mathbf{n}+\mathbf{1})}\mathbf{X}^{\mathbf{T}}_{(\mathbf{n}+\mathbf{1},\mathbf{m})}\mathbf{y}_{(\mathbf{m},\mathbf{1})}$ because unit vector.

$$\frac{\partial \mathbf{J}(\theta)}{\partial \theta} = \frac{\partial}{\partial \theta}(\theta^{\mathbf{T}}\mathbf{X}^{\mathbf{T}}\mathbf{X}\theta - 2\theta^{\mathbf{T}}\mathbf{X}^{\mathbf{T}}\mathbf{y} + \mathbf{y}^{\mathbf{T}}\mathbf{y})$$
$$= \mathbf{X}^{\mathbf{T}}\mathbf{X}\frac{\partial \theta^{\mathbf{T}}\theta}{\partial \theta} - 2\mathbf{X}^{\mathbf{T}}\mathbf{y}\frac{\partial \theta^{\mathbf{T}}}{\partial \theta} + \frac{\partial \mathbf{y}^{\mathbf{T}}\mathbf{y}}{\partial \theta}$$

Partial derivative $\frac{\partial \mathbf{x}^2}{\partial \mathbf{x}} = 2\mathbf{x}$, $\frac{\partial \mathbf{k}\mathbf{x}^2}{\partial \mathbf{x}} = \mathbf{k}\mathbf{x}$, $\frac{\partial \mathbf{Constact}}{\partial \mathbf{x}} = 0$

$$\frac{\partial \mathbf{J}(\theta)}{\partial \theta} = \mathbf{X}^{\mathbf{T}}\mathbf{X}2\theta - 2\mathbf{X}^{\mathbf{T}}\mathbf{y} + 0$$
$$0 = 2\mathbf{X}^{\mathbf{T}}\mathbf{X}\theta - 2\mathbf{X}^{\mathbf{T}}\mathbf{y}$$
$$\mathbf{X}^{\mathbf{T}}\mathbf{X}\theta = \mathbf{X}^{\mathbf{T}}$$
$$\theta = (\mathbf{X}^{\mathbf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathbf{T}}\mathbf{y}$$

Exploratory data analysis

```
#Lets look into top few rows and columns in the dataset
print(df.head())
```

Check for missing value

```
plt.figure(figsize=(12,4))
sns.heatmap(df.isnull(),cbar=False,cmap='viridis',yticklabels=False)
plt.title('Missing value in the dataset');
```

Plots

```
# correlation plot
corr = df.corr()
sns.heatmap(corr, cmap = 'Wistia', annot= True)
```



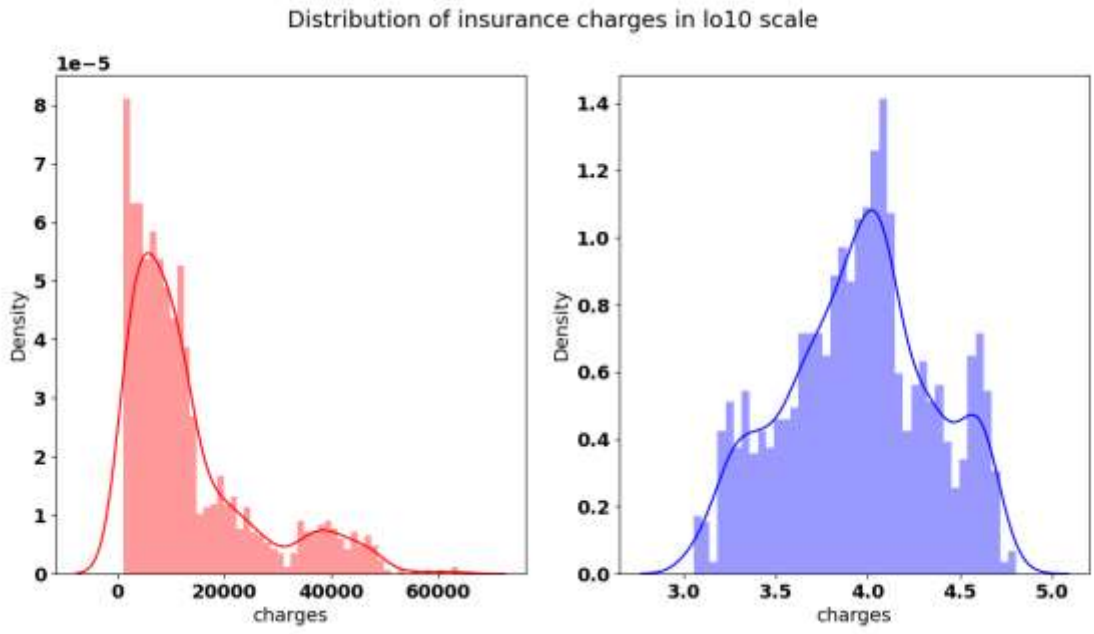
Thier no correlation among valiables.

```
f= plt.figure(figsize=(12,4))
```

```
ax=f.add_subplot(121)
sns.distplot(df['charges'],bins=50,color='r',ax=ax)
ax.set_title('Distribution of insurance charges')

ax=f.add_subplot(122)
sns.distplot(np.log10(df['charges']),bins=40,color='b',ax=ax)
ax.set_title('Distribution of insurance charges in $log$ sacle')
ax.set_xscale('log');
```

If we look at the left plot the charges varies from 1120 to 63500, the plot is right skewed. In right plot we will apply natural log, then plot approximately tends to normal. for further analysis we will apply log on target variable charges.



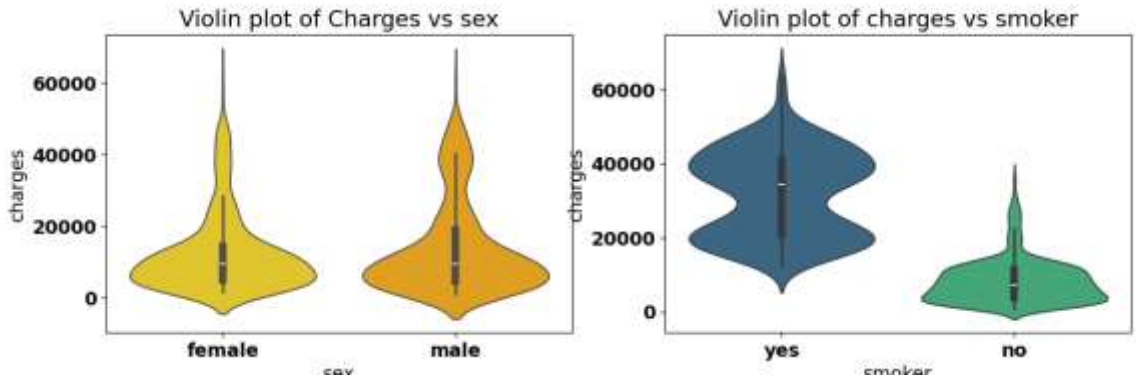
Examining the left plot reveals a range of charges spanning from 1120 to 63500, indicating a right-skewed distribution. To address this skewness, the right plot employs a natural logarithm

transformation, resulting in a plot that approximates a normal distribution. To facilitate subsequent analysis, a logarithmic transformation will be applied to the target variable, charges.

```
f = plt.figure(figsize=(14,6))
ax = f.add_subplot(121)
sns.violinplot(x='sex', y='charges',data=df,palette='Wistia',ax=ax)
ax.set_title('Violin plot of Charges vs sex')

ax = f.add_subplot(122)
sns.violinplot(x='smoker', y='charges',data=df,palette='magma',ax=ax)
ax.set_title('Violin plot of Charges vs smoker');
```

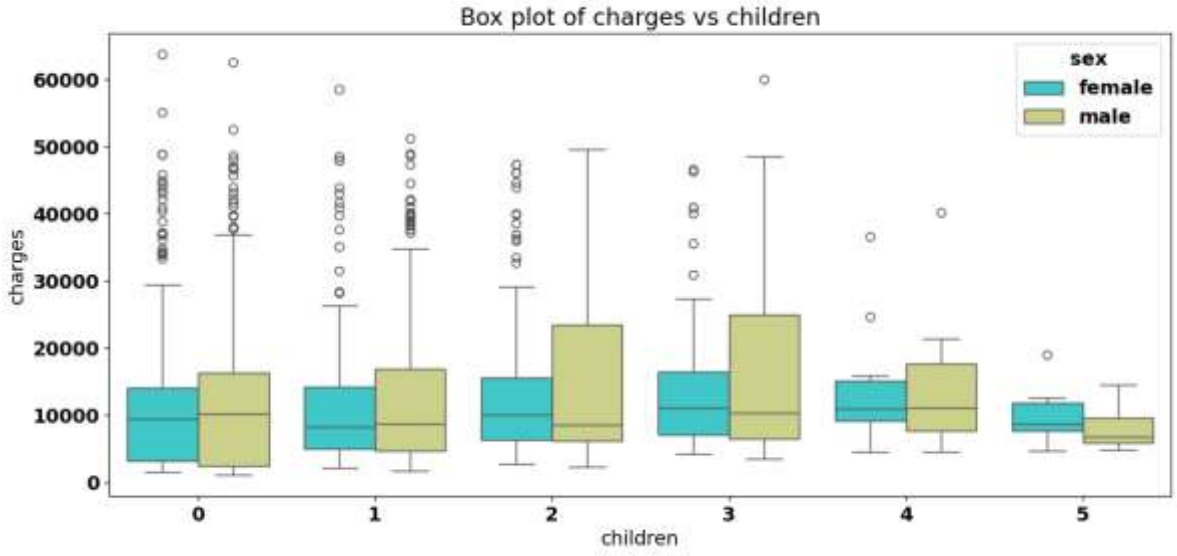
The width of the violin at each level of the categorical variable indicates the density of data points, and the shape of the violin shows the distribution of charges. This can be helpful in understanding the variability in charges for different gender groups.



In the left plot, the insurance charges for both males and females are nearly within the same range, averaging around $5000. However, in the right plot, the range of insurance charges for smokers is significantly broader compared to non-smokers. The average charges for non-smokers remain around $5000, whereas for smokers, the minimum insurance charge is set at $5000 itself.

```
plt.figure(figsize=(14,6))
```

```
sns.boxplot(x='children', y='charges',hue='sex',data=df,palette='rainbow')
plt.title('Box plot of charges vs children')
```



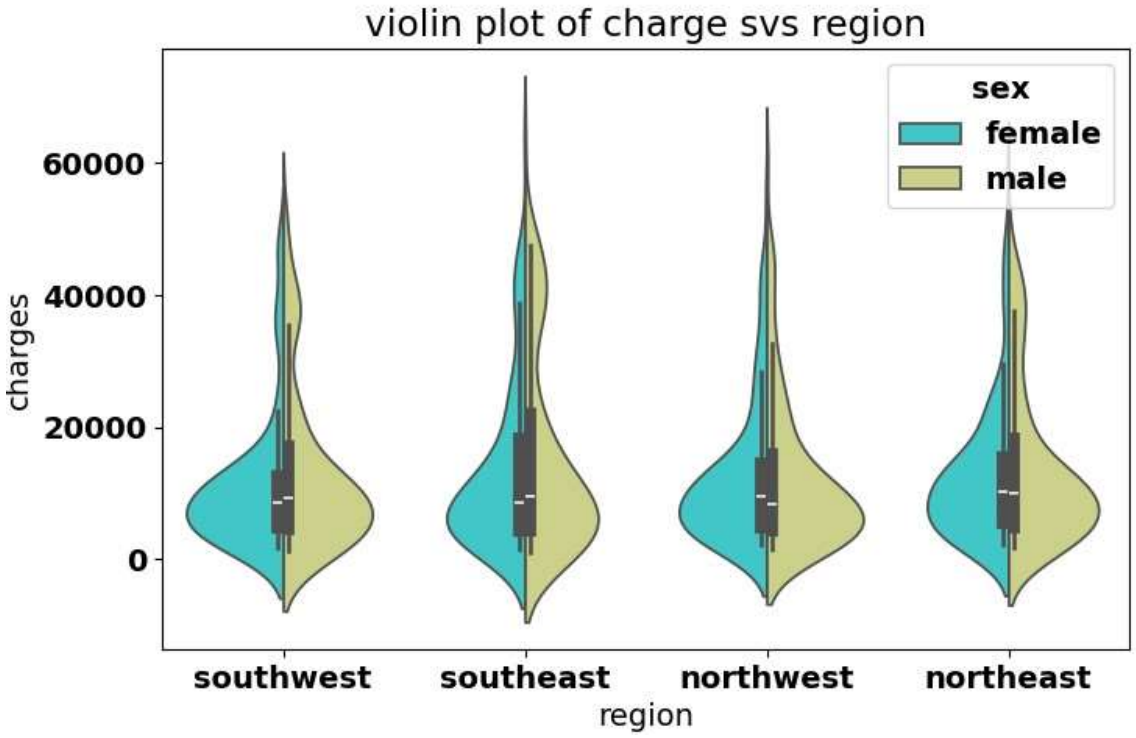
A box plot typically consists of several elements:

Box: Represents the interquartile range (IQR), with the central line inside the box indicating the median.

Whiskers: Lines extending from the box indicate the range of the data. Outliers beyond the whiskers may be individually plotted.

Outliers: Individual data points lying outside the whiskers.

```
# Group the DataFrame 'df' by the 'children' column and calculate the mean, minimum, and maximum values of the 'charges' column
grouped_stats = df.groupby('children').agg(['mean', 'min', 'max'])['charges']
plt.figure(figsize=(14,6))
sns.violinplot(x='region', y='charges',hue='sex',data=df,palette='rainbow',split=True)
plt.title('Violin plot of charges vs children');
```

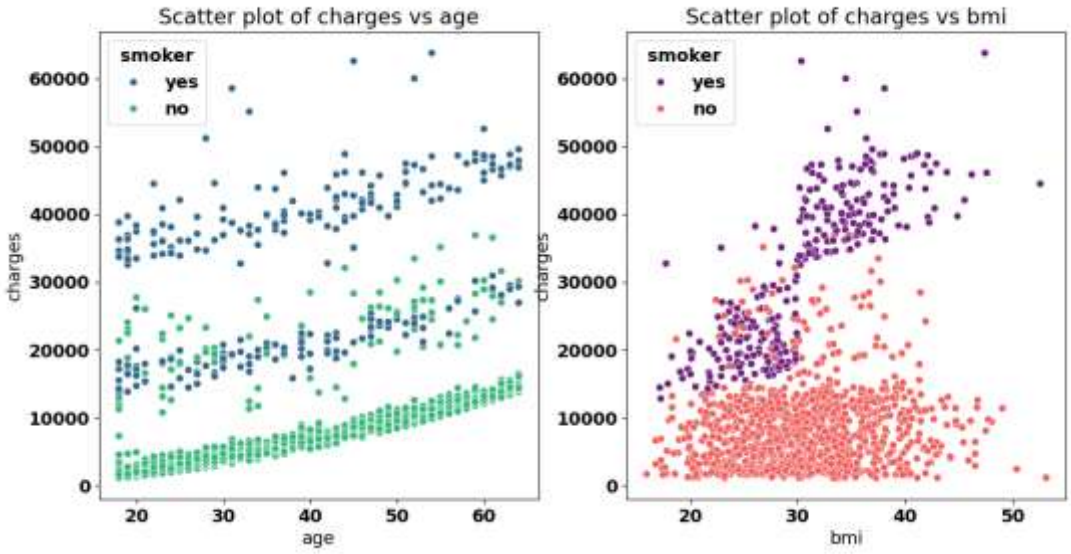



```
f = plt.figure(figsize=(14,6))
ax = f.add_subplot(121)
sns.scatterplot(x='age',y='charges',data=df,palette='magma',hue='smoker',ax=ax)
ax.set_title('Scatter plot of Charges vs age')

ax = f.add_subplot(122)
sns.scatterplot(x='bmi',y='charges',data=df,palette='viridis',hue='smoker')
ax.set_title('Scatter plot of Charges vs bmi')
plt.savefig('sc.png');
```

The scatter plot generated by this code visually explores the relationship between BMI, insurance charges, and smoking status. By using color to distinguish between smokers and non-smokers, the plot provides a clear visual representation of how these variables interact. This type of visualization is useful for identifying patterns, clusters, or trends in the data and can be

particularly effective when exploring the impact of categorical variables (like smoking status) on the relationship between two continuous variables (BMI and insurance charges).



From left plot the minimum age person is insured is 18 year. There is slabs in policy most of non smoker take 1st and 2nd slab, for smoker policy start at 2nd and 3rd slab.

Body mass index (BMI) is a measure of body fat based on height and weight that applies to adult men and women. The minimum bmi is 16kg/m^2 and maximum upto 54kg/m^2

Data Preprocessing

Encoding

To enable machine learning algorithms to effectively process data, it is imperative to convert categorical data into a numerical format.

Label Encoding

Label encoding involves the transformation of word labels into numerical representations, allowing algorithms to comprehend and operate on them.

One-Hot Encoding

One-hot encoding is a method for representing categorical variables as binary vectors, enhancing the expressiveness of the categorical data. Initially, categorical values undergo label encoding, mapping them to integer values. Subsequently, each integer value is portrayed as a binary vector with zero values except for the index corresponding to the integer, which is marked with a 1.

Dummy Variable Trap

The dummy variable trap arises when independent variables exhibit multicollinearity—a situation where two or more variables are highly correlated, meaning one variable can be predicted from the others.

By employing the pandas get_dummies function, all three aforementioned steps can be executed in a single line of code. This function is utilized to generate dummy variables for features such as sex, children, smoker, and region. Setting drop_first=True in the function aids in eliminating the dummy variable trap by dropping one variable along with the original variable. The convenience provided by pandas simplifies this crucial preprocessing step

```
# Dummy variable
categorical_columns = ['sex','children', 'smoker', 'region']
df_encode = pd.get_dummies(data = df, prefix = 'OHE', prefix_sep='_',
            columns = categorical_columns,
            drop_first =True,
           dtype='int8')
```

```
# Lets verify the dummay variable process
print('Columns in original data frame:\n',df.columns.values)
print('\nNumber of rows and columns in the dataset:',df.shape)
print('\nColumns in data frame after encoding dummy variable:\n',df_encode.columns.values)
print('\nNumber of rows and columns in the dataset:',df_encode.shape)
```

Box-Cox Transformation:

The Box-Cox transformation is a method used to convert non-normally distributed dependent variables into a more normal distribution. Achieving normality is crucial for various statistical techniques, and by applying a Box-Cox transformation, you expand the range of tests that can be performed on your data. To execute this transformation, it is necessary to determine the lambda value and then apply the specified rule to the variable in question.

$$\begin{cases} \frac{y^{\lambda}-1}{\lambda}, & y_i \neg = 0 \\ log(y_i) & \lambda = 0 \end{cases}$$

The trick of Box-Cox transformation is to find lambda value, however in practice this is quite affordable. The following function returns the transformed variable, lambda value,confidence interval

```
from scipy.stats import boxcox
y_bc,lam, ci= boxcox(df_encode['charges'],alpha=0.05)


#df['charges'] = y_bc
# it did not perform better for this model, so log transform is used
ci,lam
```

The code snippet you provided is using the Box-Cox transformation from the **scipy.stats** module to transform the 'charges' variable in your dataset. Let's break down the key elements

boxcox: This function performs the Box-Cox transformation on the specified variable. In this case, it's applied to the 'charges' column of the DataFrame (df_encode).

df_encode['charges']: This selects the 'charges' column from the DataFrame.

alpha=0.05: The alpha parameter specifies the confidence level for the confidence interval (ci) calculation. In this case, a 95% confidence interval is used (alpha=0.05).

y_bc: The transformed data after applying the Box-Cox transformation.

lam: The lambda parameter determined by the transformation. The lambda value is an important parameter in the Box-Cox transformation, as different values of lambda can be applied to achieve the best normality.

ci: The confidence interval of the lambda parameter.

```
## Log transform
df_encode['charges'] = np.log(df_encode['charges'])
```

Train Test split

In [17]:

```
from sklearn.model_selection import train_test_split
X = df_encode.drop('charges',axis=1) # Independet variable
y = df_encode['charges'] # dependent variable

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3,random_state=23
```

The original categorical variable are remove and also one of the one hot encode varible column for perticular categorical variable is droped from the column. So we completed all three encoding step by using get dummies function.

Train Test split

```
from sklearn.model_selection import train_test_split
X = df_encode.drop('charges',axis=1) # Independet variable
y = df_encode['charges'] # dependent variable
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3,random_state=23)
```

Model building

In this step build model using our linear regression equation $\theta = (\mathbf{X}^{\mathbf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathbf{T}}\mathbf{y}.$

In first step we need to add a feature x0=1 to our original data set.

```
# The parameters for linear regression model
parameter = ['theta_'+str(i) for i in range(X_train_0.shape[1])]
columns = ['intersect:x_0=1'] + list(X.columns.values)
parameter_df = pd.DataFrame({'Parameter':parameter,'Columns':columns,'theta':theta})
```


```
# The parameters for linear regression model
parameter = ['theta_'+str(i) for i in range(X_train_0.shape[1])]
columns = ['intersect:x_0=1'] + list(X.columns.values)
parameter_df = pd.DataFrame({'Parameter':parameter,'Columns':columns,'theta':theta})
```


```
# Scikit Learn module
from sklearn.linear_model import LinearRegression
lin_reg = LinearRegression()
lin_reg.fit(X_train,y_train) # Note: x_0 =1 is no need to add, sklearn will take care of it.

#Parameter
sk_theta = [lin_reg.intercept_]+list(lin_reg.coef_)
parameter_df = parameter_df.join(pd.Series(sk_theta, name='Sklearn_theta'))
parameter_df
```

The parameter obtained from both the model are same.So we succefull build our model using normal equation and verified using sklearn linear regression module. Let's move ahead, next step is prediction and model evaluvation.

Model evaluation

We will predict value for target variable by using our model parameter for test data set. Then compare the predicted value with actual valu in test set. We compute **Mean Square Error** using formula

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

R^2 is statistical measure of how close data are to the fitted regression line. R^2 is always between 0 to 100%. 0% indicated that model explains none of the variability of the response data around it's mean. 100% indicated that model explains all the variablity of the response data around the mean.

$$R^2 = 1 - \frac{SSE}{SST}$$

SSE = Sum of Square Error

SST = Sum of Square Total

$$SSE = \sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

$$SST = \sum_{i=1}^{m}(y_i - \bar{y}_i)^2$$

```
# Normal equation
y_pred_norm =  np.matmul(X_test_0,theta)


#Evaluvation: MSE
J_mse = np.sum((y_pred_norm - y_test)**2)/ X_test_0.shape[0]


# R_square
sse = np.sum((y_pred_norm - y_test)**2)
```

```
sst = np.sum((y_test - y_test.mean())**2)
R_square = 1 - (sse/sst)
print('The Mean Square Error(MSE) or J(theta) is: ',J_mse)
print('R square obtain for normal equation method is :',R_square)
```

```
The Mean Square Error(MSE) or J(theta) is:  0.18729622322982067
R square obtain for normal equation method is : 0.7795687545055299
```

```
#Evaluvation: MSE
from sklearn.metrics import mean_squared_error
J_mse_sk = mean_squared_error(y_pred_sk, y_test)
```

```
# R_square
R_square_sk = lin_reg.score(X_test,y_test)
print('The Mean Square Error(MSE) or J(theta) is: ',J_mse_sk)
print('R square obtain for scikit learn library is :',R_square_sk)
```

```
The Mean Square Error(MSE) or J(theta) is:  0.1872962232298189
R square obtain for scikit learn library is : 0.7795687545055318
```

The model returns R^2 value of 77.95%, so it fit our data test very well, but still we can imporve t he the performance of by diffirent technique. Please make a note that we have transformer out va riable by applying natural log. When we put model into production antilog is applied to the equat ion.

2.INTRODUCTION TO DEEPLEARNING

Deep learning algorithms use ANNs (short for Artificial Neural Networks) to duplicate the functioning of our brain. Why though? Not trying to be philosophical or historical, but our brain

is the most complicated piece of machinery we know. Due to this, even if we can replicate the 'brain-power' the number of possibilities is unfathomable.

How an algorithm processes and computes the output is of course different from another algorithm. For example, in one type of algorithm MLPs, the data is processed by the stacking the basic form of neural networks (also called a perceptron) on top of each other and at different levels. The data is divided between these sections, processed, and an output or multiple outputs are generated.

- **Convolutional Neural Networks (CNNs)**
- **Recurrent Neural Networks (RNNs)**
- **Long Short-Term Memory Networks (LSTMs)**
- **Multilayer Perceptrons (MLPs)**
- **Radial Basis Function Networks (RBFNs)**
- **Generative Adversarial Networks (GANs)**
- **Deep Belief Networks (DBNs)**
- **Self-Organizing Maps (SOMs)**
- **Autoencoders**
- **Restricted Boltzmann Machines (RBMs)**

Convolutional Neural Networks

Working: They have multiplayer layers that process the features from the data.
These networks are mostly used for detecting objects and processing images.

Important layers that carry out the output:

Convolutional Layer: The layer has multiple filtering operations that carries out complex operations.
Rectified Linear Unit: Also called ReLU for short. This unit applies some operations on the elements and outputs a rectified feature map.

Pooling Layer: Output of ReLU goes into this Pooling Layer. Pooling is done to reduce the dimensionality of the feature map.

The pooling layer outputs a single long vector from the 2-D output of ReLU.

If we are using CNNs for image classification, for example, then in the end, after pooling is done, the vector from pooling layer goes into a feed forward network or a fully connected layer which will classify the images.

Recurrent Neural Networks (RNNs) utilize their outputs as inputs, allowing them to retain information from prior inputs. This characteristic enables RNNs to execute potent operations on data by leveraging their memory of past inputs. The self-feeding property of RNNs also permits them to unfold over time, signifying that the output at one time step (t1) can differ from the output at another time step (t2).

Used for:

- Image Captioning
- NLP (Natural Language Processing)
- Time Series Analysis
- Language Translation

Long Short-Term Memory Networks (LSTMs)

These are a type of RNNs that has the default behaviour of recalling past information (sounds human enough!) for quite long periods.

The use cases of LSTMs are very similar to RNNs but more specifically, they can be used for music composition, speech recognition, etc.

Working of an LSTM is carried out in three steps:

- Forget the unnecessary information of the past state.
- Update the current cell values.
- Output the certain state of the cell.

Mutlilayer Perceptrons (MLPs)

If you know about the perceptron, then you would have guessed how this algorithm work just from reading the title.

They contain various perceptrons with activation functions stacked vertically and connected horizontally.

MLPs are used for speech-recognition, translation, and speech-recognition.

So, those were the most popular neural network algorithms. If you want to know more in detail about the networks described above, and the others.